



Универзитет „Св. Кирил и Методиј“ во Скопје
ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

ДИПЛОМСКА РАБОТА

Мобилна апликација за решавање на програмски задачи

Изработил:

Мартин Трајковски индекс: 195063

Ментор:

проф. Д-р Магдалена Костоска

Скопје, 25 Фев 2026

Тема:	**Мобилна апликација за решавање на програмски задачи**
Автор:	**Мартин Трајковски**
Научна област:	**Информатика**
Ментор:	**проф. Д-р Магдалена Костоска**
Датум на одбрана:	
Членови на комисија:	**проф. Д-р Магдалена Костоска** Факултет за информатички науки и компјутерско инженерство **проф. Д-р Христина Михајлоска** Факултет за информатички науки и компјутерско инженерство **проф. Д-р Панче Рибарски** Факултет за информатички науки и компјутерско инженерство

Апстракт

Апликацијата е наменета за решавање на програмски задачи со фокус на задачи од областа на алгоритми и податочни структури.

Корисниците ќе имаат можност интерактивно да ги вежбаат и тестираат своите знаења преку задачи, со што ќе ги подобруваат своите програмерски вештини. Притоа оваа апликација може да се користи за да се олесни припремата за интервју за пракса или работа.

Системот се состои од два делови. Првиот дел се состои од решавање на задачи, при што решенијата ќе се оценуваат автоматски и системот обезбедува повратни информации. Вториот дел од системот е форум кој ќе овозможи корисникот да поставува прашања и дава одговори.

Содржина

ВОВЕД **5**

Што се алгоритми и податочни структури? 5
Зошто се важни алгоритмите и податочните структури? 5
Компетитивно програмирање 6

ВОВЕД ВО АПЛИКАЦИЈАТА **7**

Цел на апликацијата 7
Структура на апликација 7
Модул за решавање на задачи и натпревари *7*
Форум за дискусија и соработка *8*
Карактеристики на апликацијата 8

ОПИС НА КОРИСТЕНА ТЕХНОЛОГИЈА **8**

ОПИС НА ТЕХНИЧКО РЕШЕНИЕ **13**

Архитектура 13
Фронтенд слој *14*
Proxy слој *15*
Backend слој *15*
Слој за управување со идентитети, автентикација и авторизација (IAM) *15*
Бази на податоци *15*
Дизајн на базите на податоци 16
Дизајн на базата на податоци за модулот за натпревари и задачи *16*
Дизајн на базата на податоци за форум модулот *17*
Дизајнирање на Апликациски програмски интерфејс со репрезентативен пренос на состојба 18
JSON и HAL одговор (application/json+hal) *20*
Безбедност во апликациите 21
Контејнеризација и градење на апликацијата 22
Compose датотека *23*

ПРЕГЛЕД НА АПЛИКАЦИЈА **25**

Најава или регистрација на корисник 25
Поставки во апликацијата 26
Задачи (Проблеми) 27
Филтрирање, сортирање, поглед *28*
Арена *28*
Натпревари 29
Форум 30
Заедница *30*
Користење на Админ апликација 31
Филтрирање и преглед на податоци *32*
Креирање и ажурирање на категории *33*
Креирање на задачи *34*
Креирање на натпревари *34*
Преглед на форум податоци и нивно креирање *36*
Креирање на заедница *37*

РЕФЕРЕНЦИ **39**

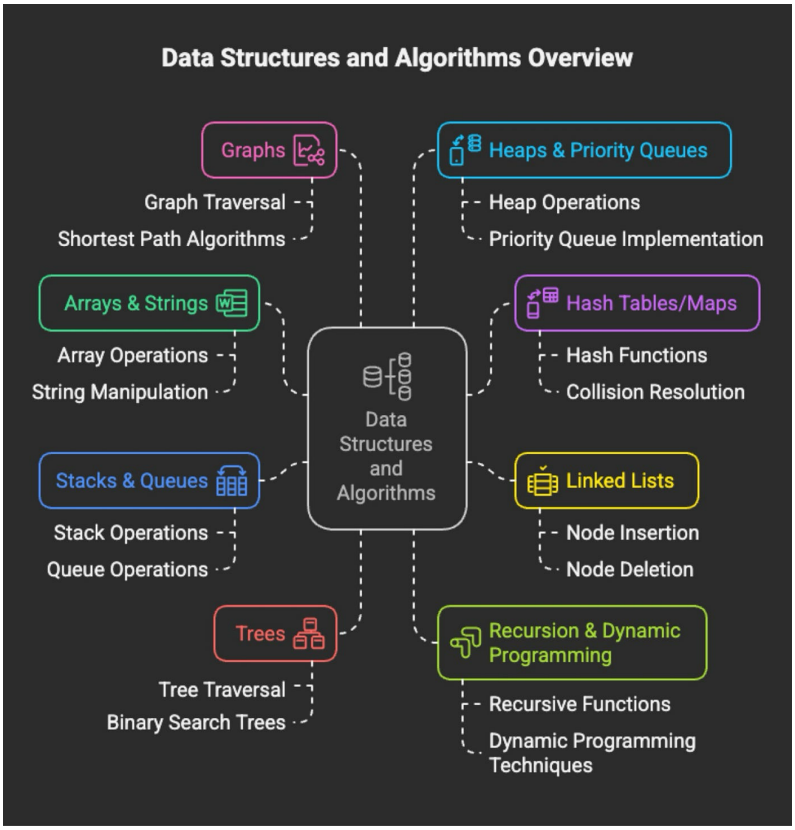
Вовед

Во денешниот дигитален свет, каде што технологијата постојано напредува и влијае на секој аспект од нашето секојдневие, способноста за ефективно решавање на проблеми преку програмирање е од суштинско значење. Алгоритмите и податочните структури се темелни концепти во компјутерските науки, кои не само што се основа за развој на софтверски решенија, туку и клучни се за оптимизација на перформансите, ефикасноста и скалабилноста на апликациите.

Што се алгоритми и податочни структури?

Алгоритмите се јасно дефинирани сетови на инструкции кои опишуваат како да се изврши одредена задача или да се реши проблем. Тие се основа на секој компјутерски процес и се користат во најразлични апликации, од пребарување на информации до обработка на големи податоци.

Податочните структури, пак, се начини на организирање и складирање на податоци така што тие да можат да се користат ефективно. Изборот на соодветна податочна структура е клучен за оптимизација на перформансите на апликацијата, бидејќи различни структури имаат различни карактеристики и се погодни за различни задачи.



Слика 1 Вовед во податочни структури и алгоритми *[24]*

Зошто се важни алгоритмите и податочните структури?

Алгоритмите и податочните структури се основа на современото програмирање и развој на софтвер. Тие не само што овозможуваат решавање на комплексни проблеми, туку и оптимизација на перформансите, ефикасноста и скалабилноста на апликациите. Разбирањето и примената на овие концепти се од клучно значење за секој програмер и развивач, и

претставуваат темел за успешна кариера во технологијата.

1. Ефикасност и перформанси

Изборот на соодветен алгоритам и податочна структура може значително да ја подобри ефикасноста на апликацијата. На пример, користењето на бинарно пребарување наместо линеарно пребарување може да го намали времето на извршување од O(n)[1] на O(log n), што е особено важно при работа со големи количини на податоци.

2. Решавање на комплексни проблеми

Многу од современите апликации се соочуваат со комплексни проблеми кои бараат напредни алгоритми и податочни структури за нивно решавање. На пример, алгоритмите за машинско учење и обработка на природен јазик се темелат на напредни математички концепти и податочни структури кои овозможуваат ефикасна обработка на големи количини на податоци.

3. Подготовка за технички интервјуа

Компании како Google, Amazon, Microsoft и други технолошки гиганти често користат алгоритми и податочни структури како основа за проценка на кандидатите во техничките интервјуа. Разбирањето на овие концепти не само што помага во решавање на проблемите, туку и демонстрира способност за логичко размислување и аналитичко решавање на проблеми.

4. Оптимизација на ресурсите

Во денешно време, кога ресурсите како меморија и процесорска моќ се ограничени, оптимизацијата на кодот е од клучно значење. Алгоритмите и податочните структури овозможуваат развој на софтверски решенија кои се ефикасни во користењето на овие ресурси, што е особено важно во мобилните апликации и апликациите за интернет на нештата (IoT).

5. Развој на софтверски решенија

Разбирањето на алгоритмите и податочните структури овозможува развој на софтверски решенија кои се не само функционални, туку и ефикасни и скалабилни. Ова е особено важно во областа на веб развојот, каде што апликациите треба да бидат способни да се справат со голем број на корисници и големи количини на податоци.

Компетитивно програмирање

Во современиот дигитален свет, каде технологијата е движечка сила на иновациите, компетитивното програмирање се наметнува како интелектуално натпреварување кое ги спојува логиката, математиката и вештината на кодирање во една дисциплина. За многу млади ентузијасти, студенти и професионалци, ова претставува повеќе од обично учење – тоа е арена во која се натпреваруваат идеи, стратегии и прецизност. Како што спортот ја тестира физичката издржливост, така и компетитивното програмирање ги тестира менталните способности преку решавање комплексни задачи на ограничено време. Натпревари како Олимпијадата по информатика, ICPC, Google Code Jam, и други, претставуваат предизвик и мотивација за континуиран напредок, каде учесниците се борат не само за медали, туку и за личен раст, признавање и развој на вештини кои се основа на модерното софтверско инженерство.

[1] O(n) – Големо О нотација која покажува како времето на извршување или искористеноста на меморија расте со растењето на влезот

Вовед во Апликацијата

Брзото и ефикасно решавање на програмски задачи претставува клучна вештина за успех во современата ИТ индустрија. Алгоритмите и податочните структури се основа на секој софтверски систем, а нивното длабоко разбирање е неопходно за креирање на квалитетни и оптимизирани решенија.

Оваа апликација е развиена со цел да им овозможи на корисниците интерактивно да ги подобрат своите програмерски способности преку учење и практична работа. Мотивацијата зад нејзината изработка произлегува од љубовта кон програмирањето и искуствата стекнати преку користење на слични платформи. Таа е наменета за сите програмери од почетници до искусни, кои сакаат да ги усовршат своите знаења или да се подготват за технички интервјуа.

Со користење на оваа апликација, корисниците ќе имаат можност да развијат критично размислување, да се справуваат со комплексни проблеми и да се стекнат со практични вештини релевантни за реалните предизвици во ИТ секторот.

Цел на апликацијата

Целта на оваа апликација е да создаде структурирана, интерактивна средина за усовршување на алгоритамските и програмерските вештини. Преку внимателно дизајнирани функционалности, таа овозможува систематско решавање на задачи, добивање повратна информација, споредување решенија и учество во натпревари. Наместо пасивно учење, апликацијата промовира активен пристап, каде секој корисник може да напредува според сопственото темпо и интереси.

Преку ваков пристап, апликацијата има за цел не само да го олесни стекнувањето на техничко знаење, туку и да ја поттикне логичката мисла, аналитичките способности и самостојната работа, клучни компоненти за успех во реални програмерски ситуации. Таа служи како мост меѓу теоретското знаење и практичната примена, нудејќи им на корисниците алатка што ќе им помогне да се истакнат во професионалното поле.

Структура на апликација

Апликацијата е поделена на два главни модули, со цел да се организира логиката и функционалноста на системот на појасен и модуларен начин.

Модул за решавање на задачи и натпревари

Овој модул има за цел да овозможи на корисникот да решава задачи, учествува во натпревари итн, а како фунционалности ги содржи следните точки:

- Админ може да креира на категории, задачи, натпревари и да ги ажурира или бриши од базата на податоци
- Избор на задачи по ниво на тежина (лесни, средни, тешки).
- Тематска класификација на задачите: сортирање, графови, дрва, динамичко програмирање и сл.
- Решавање задачи со примена на соодветни алгоритми и податочни структури.
- Тестирање на код со различни влезни податоци, вклучително и гранични случаи.

- Автоматска проверка и повратна информација во реално време.
- Учествување во натпревари и следење на рангирања.
- Следење на лична статистика според број и тежина на решени задачи или избран програмски јазик.

Форум за дискусија и соработка

Овој модул има за цел да ја поттикне соработката и размената на знаење помеѓу корисниците и како фунцтионалности ги содржи:

- Админ можи да креира на категории, заедници, постови, да ги ажурира или брише од базата на податоци
- Админ можи да додава модератори на заедници
- Админ можи да прегледува постови на заедници и да ги маркира во случај да се несоодветни
- Корисниците можат да поставуваат прашања и да добиваат одговори од други членови на заедницата.
- Споделување на решенија и стратегии за решавање на задачи.
- Размена на искуства и совети за подготовка за интервјуа.
- Поддршка и мотивација од заедницата на програмери.

Карактеристики на апликацијата

Интерактивно учење: Корисниците активно се вклучуваат во процесот преку решавање задачи и анализа на решенија.
Автоматско оценување: Системот автоматски ги проверува поднесените решенија од играчите и обезбедува прецизни повратни информации.
Поддршка за заедницата: Преку форумот и натпреварите, се поттикнува соработка и градење на активна програмерска заедница.

Опис на користена технологија

За развој на оваа веб-апликација се користат современи технологии кои овозможуваат брза, ефикасна и интуитивна работа. Најглавни технологии во системот се Spring Boot [1] кој овозможува непречена работа на апликацијата и нуди добра архитектура и организација на кодот, и Flutter [2] со Dart [3] кој овозможува креирање на мобилната апликација која ќе може корисникот да ја инсталира на неговата машина. Исто така и другите користени технологии се битни во случајот бидејќи овозможуваат апликацијата да работи како целина заедно со сите други сервиси. Технологиите кои се користат за креирање во целата апликацијата се:

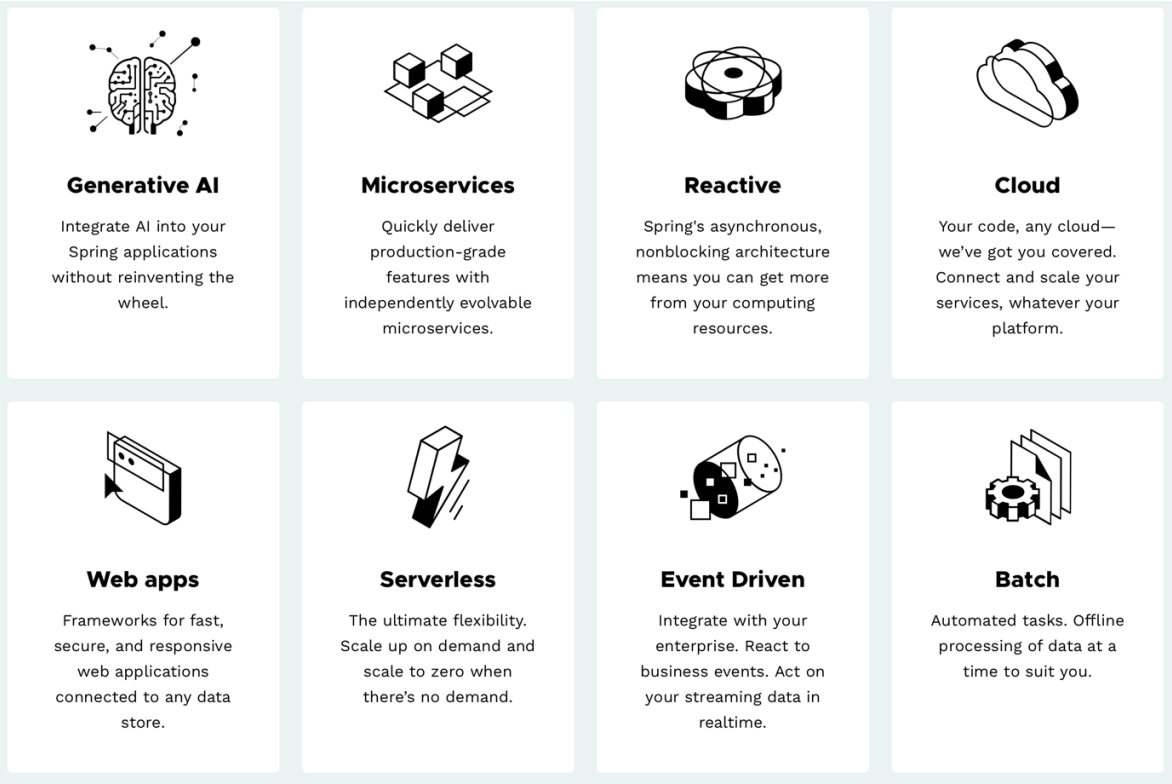


Spring Framework

Spring Framework [4] е една од најпопуларните изградени рамки и најшироко користена рамка за развој на апликации. Обезбедува инфраструктура базирана на програмскиот јазик Java [5], со цел да се поедностави процесот на развој, полесно тестирање и одржување на изграденото софтверско решение. Spring Framework е решение дизајнирано со отворен код кое промовира програмски практики како што се модуларност, слаба поврзаност (loose coupling) и повторна употребливост на код. Двата клучни основни концепти врз кои е базирана рамката се Inversion of Control кој е софтверски дизајн на принцип кој ја пренесува

работата на креирање, конфигурација и животниот тек на објект на екстерна рамка или контејнер и Dependency Injection (DI) кое овозможува кога ќе се креира еден објект тој да ги добие своите зависни објекти од екстерен извор без да ги рекреира истите. Овие два концепти работат взаемно така што при креирање на една инстанца од објект се користи техника на инјектирање на зависности на тој објект и најчесто се прави преку конструктор. Рамката е специјално дизајнирана и најчесто се користи за брз и стабилен развој на веб апликации и микросервиси.

Главни предности на Spring Framework е неговата интеграција со други технологии рамки како што се Hibernate [6], Java Persistence API (JPA), RESTful [7] веб сервиси, разни типови на бази на податоци и многу други. Ова го прави Spring да биде погоден за развој на сложени и скалабилни апликации.
Со надградба на оваа рамка изграден е Spring Boot кој автоматски може да ги конфигурира компонентите или интеграции и функционалности кој ги овозможува Spring рамката така што на развивачите им овозможува повеќе да се фокусираат на имплементирање и тестирање на бизнис логиката ослободувајќи ги од сложени инфраструктурни задачи.



Слика 2 Дел функционалности на Spring рамката (Spring Framework *[4]*)

Flutter

Flutter [2] претставува современ софтверски прибор од алатки за развој на кориснички интерфејс, развиен од компанијата Google, кој овозможува изработка на мултиплатформски апликации за Android, iOS, Web и десктоп околини преку користење на единствена кодна база. Основата на Flutter е програмскиот јазик Dart [3], исто така развиен од Google, кој поддржува објектно-ориентирано и реактивно програмирање.

Во рамките на Flutter, изворниот код се компајлира директно во нативен машински код, што

овозможува високи перформанси, намалена латентност и ефикасно извршување на апликациите на различни платформи. Во апликациите развиени со Flutter, секој елемент на корисничкиот интерфејс е моделиран како виџет (widget), што овозможува високо ниво на флексибилност, модуларност и повторна употребливост на софтверските компоненти, како и едноставно прилагодување и проширување на корисничкиот интерфејс. Дополнително, Flutter користи сопствен рендеринг механизам базиран на графичкиот engine Skia, кој овозможува исцртување на визуелните елементи независно од нативните компоненти на оперативниот систем. Овој пристап обезбедува конзистентен визуелен приказ и стабилни перформанси на сите поддржани платформи, без оглед на специфичните карактеристики на околината во која апликацијата се извршува.



Keycloak

Keycloak [8] е моќна платформа со отворен код за управување со идентитети, автентикација и авторизација развиена од Red Hat. Тој овозможува централизирано решение за автентикација и авторизација на корисници во апликации и сервиси и е погодна алатка за системи каде безбедноста и контролата на пристап се критични. Keycloak ја олеснува имплементацијата да безбедносни механизми и ја намалува потребата од рачно развивање на комплексна логика за најава, регистрација и управување со кориснички улоги и поддржува повеќе стандардни протоколи вклучувајќи ги и OAuth2.0 [9], OpenID Connect (OIDC) [10] кој е надградба на OAuth2.0 и SAML 2.0, [11] што им овозможува на развивачите интеграција со различни типови на апликации. Една од главните функционалности на Keycloak е Single Sign-On (SSO) што им овозможува на корисниците да се најават еднаш и да добијат пристап до повеќе апликации без повторна најава. Обезбедува богати можности за управување со корисници како дефинирање на улоги и дозволи, дефинирање на политики за лозинки, интеграција со LDAP и Active Directory и многу други функционалности.

Често се користи во модерните апликации како централен авторизациски сервис кој издава JSON Web Token (JWT) [12], кој се користат од клиентските и ресурсните апликации за верификација на идентитетот и правата на корисникот. Особено е погоден за микросервисни архитектурни апликации.



Nginx

Nginx [13] е високо перформантен веб сервер со отворен код, кој е широко користен во современите веб архитектури. Покрај неговата улога примарна улога како веб сервер, често се користи и како **proxy**, односно како посредник помеѓу клиентски апликации и ресурсни сервери. Во оваа улога Nginx прима барања од клиентите и ги проследува дојдовните барања до соодветните сервиси, при што се апстрахира архитектурата на системот. Користењето на Nginx како **Reverse Proxy** овозможува подобрување на безбедноста и скалабилноста на апликацијата. Клиентите комуницираат исклучиво со Nginx додека ресурсните сервери или авторизациски сервиси остануваат скриени од директен пристап така што се намалува изложеноста на системот од потенцијалени безбедносни закани. Дел од функционалностите кој ги нуди Nginx како Reverse Proxy се **Load Balancing** кој овозможува рамномерна распределба на барањата кон повеќе еднакви ресурсни инстанци од еден сервис, **SSL/TLS Termination** кој овозможува енкриптирана комуникација со клиентите, додека внатрешната

комуникација со ресурсните сервиси може да се одвива без енкрипција, со што се подобруваат перформансите, кеширање и многу други функционалности.

Во рамки на архитектурата, Nginx се користи како Reverse Proxy компонента која ги пренасочува дојдовните HTTP(S) [14] барања до соодветните сервиси. Како резултат, системот добива зголемена сигурност, стабилност и скалабилност.



Go (Golang)

Програмскиот јазик Go [15], познат и како Golang, е развиен од компанијата Google со цел да се поедностави развојот на ефикасни, скалабилни и сигурни софтверски системи, особено во доменот на серверски, мрежни и дистрибуирани апликации. Јазикот е дизајниран да комбинира јасна и минималистичка синтакса со високи перформанси и вградена поддршка за конкурентно програмирање преку лесни нишки на извршување, што овозможува ефикасно искористување на повеќејадрените процесорски архитектури. Go користи компајлиран модел на извршување, при што изворниот код се преведува директно во машински код, со што се постигнува брзо извршување, намалена потрошувачка на системски ресурси и генерирање на самостојни бинарни извршни датотеки, кои ја олеснуваат дистрибуцијата и деплојувањето на апликациите без дополнителни зависности. Дополнително, јазикот обезбедува вградени механизми за управување со меморија, автоматско собирање на отпад (garbage collection) и богата стандардна библиотека со поддршка за мрежна комуникација и работа со веб-протоколи. Поради наведените карактеристики, Go е широко применуван во развојот на микросервисни архитектури, каде што неговата ефикасност, едноставност и оптимизирано управување со ресурси овозможуваат лесна интеграција со технологии за контејнеризација, како што е Docker [16], како и стабилна имплементација на високо-достапни и скалабилни системи.

Во рамки на архитектурата, со програмскиот јазик Go е креирана апликација која служи како посебно Reverse Proxy со цел да се одвои Административниот дел за управување со ресурси од другите сервиси во архитектурата.



Svelte.js

Svelte.js [17] е претставува современа JavaScript [18] рамка со отворен код, наменета за развој на интерактивни и високо перформантни клиентски веб апликации. За разлика од традиционалните рамки за развој на веб апликации кои ја извршуваат најголемата логика во прелистувачот, оваа рамка применува пристап на компајлирање на кодот, при што апликацијата се трансформира во оптимизиран JavaScript код за време на процесот на градење. Клучна карактеристика на Svelte е отсуството на виртуелниот Document Object Model (DOM) така што наместо тоа, Svelte директно генерира код кој прецизно ги ажурира елементите на корисничкиот интерфејс кога ќе настане промена во состојбата на апликацијата. Овој пристап резултира со подобри перформанси, помала големина на финалниот bundle.js и поефикасно користење на ресурсите на клиентската страна. Svelte исто така нуди едноставна декларативна синтакса, природна реактивност и лесно може да се интегрира со разни технологии и RESTful сервиси што го прави погоден за користење како frontend слој во модерни софтверски архитектури.

Во рамки на архитектурата се користи за градење на клиентска апликација која ги поврзува сервисите за администрација и управување со ресурси во апликацијата и поврзување и најава преку сервисот за управување со идентитети, автентикација и авторизација.



Docker and Docker Compose

Docker [16] претставува платформа со отворен код за контејнеризација на апликациите заедно со нивните зависности во изолирани контејнери. Овој пристап обезбедува конзистентно извршување на апликациите на различни околини, независно од оперативниот систем или инфраструктурата. Docker значително го поедноставува процесот на развој, тестирање и деплојување на софтверските решенија што го прави да биде погоден за микросервисни архитектури и системи во облак. Основниот концепт на Docker се базира на користење на контејнери, кои претставуваат лесни и изолирани извршни околини. За креирање на еден контејнер се користи слика таканаречена Docker Image која содржи се што е потребно за извршување на апликацијата. Процесот на дефинирање на слика се дефинира преку **Dockerfile**, што овозможува автоматизирано градење на апликациите.

Docker Compose е алатка која овозможува дефинирање и управување со повеќе поврзани Docker контејнери како една целина. Сите сервиси и конфигурации се дефинираат со една датотека наречена **docker-compose.yml** која ги содржи нивните слики, мрежи, дискови, мрежна конфигурација и меѓусебни зависности. Docker Compose е особено корисен за локален развој и тестирање, каде што е потребно истовремено градење и стартување на повеќе поврзани компоненти.

Со користење на Docker и Docker Compose се добива подобра организација и олесната соработка меѓу членовите во тимот. Овие технологии претставуваат стандарден дел од современите DevOps практики и имаат значајна улога во автоматизација на процесите за развој и деплојување на софтверските решенија.



Neo4j

Neo4j [19] претставува граф-ориентирана база на податоци со отворен код, наменета за складирање, управување и обработка на податоци кои се меѓусебно поврзани. За разлика од релационите бази на податоци, каде што податоците се организираат во табели, Neo4j користи **граф модел** кој ги моделира податоците како јазли, релации и својства, што овозможува природно претставување на комплексни поврзаности. Овој модел е особено корисен во случаи како системи за препораки, социјални мрежи и анализа на поврзаност. Основната предност е способноста за брзо и ефикасно извршување на пребарувања кои вклучуваат повеќекратни релации. Операциите како што се пронаоѓање на патеки, анализа на мрежи и детекција на обрасци се значително побрзи во споредба со традиционалните релациони системи. За пребарување Neo4j го користи својот сопствен декларативен јазик Cypher Query Language (CQL), кој овозможува интуитивно и моќно пребарување низ мрежи од податоци, со високи перформанси дури и кај сложени структури. Neo4j исто така поддржува и ACID трансакции, што обезбедува конзистентност и сигурност на податоците, при пристап на податоците од страна на повеќе корисници и нуди можности за скалабилност

и репликација како традиционалните релациони бази на податоци. Поради флексибилноста, моќните можности за анализа на граф податоци и активната заедница, Neo4j претставува значаен избор за системи кои бараат ефикасна обработка на поврзани податоци.



PostgreSQL

PostgreSQL [20] претставува релациона база на податоци со отворен код, позната е по својата стабилност, сигурност и напредни можности за управување со податоци. Дизајнирана е да поддржува комплексни апликации и високи барања на конзистентност и интегритет на податоците. PostgreSQL го следи релациониот модел на податоци, при што информациите се организирани во табели составени од редови и колони и обезбедува целосна поддршка за Structured Query Language (SQL) стандарди како и напредни функционалности како тригери, погледи, складирани процедури и друго. За оваа база на податоци исто така може да се спомене дека поддржува ACID трансакции при што обезбедува атомичност, конзистентност, изолираност и трајност на податоците, што ја прави оваа база на податоци погодна за апликации каде што сигурноста и точноста на податоците се од клучно значење, како што се финансиски системи и деловни апликации. Исто така битно е да споменеме дека нуди и механизми за индексирање, што овозможува подобри перформанси при пребарување и анализа на податоци и поддржува и работа со JSON и JSONB и многу други функционалности. Поради својата стабилност, флексибилност и база на податоци со отворен код, често се избира како алтернатива на комерцијални бази на податоци како Oracle и Microsoft SQL Server.

Опис на техничко решение

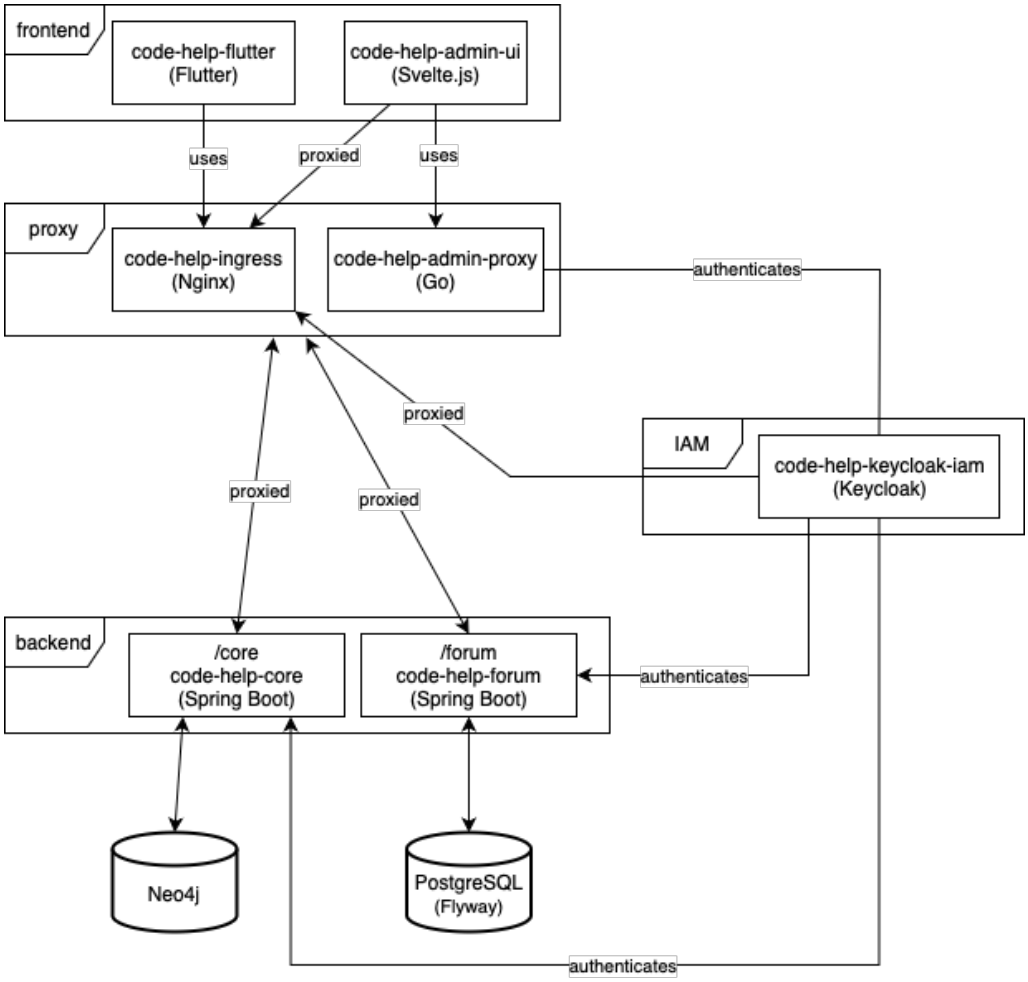
Во рамките на ова техничко решение е анализиран и разработен модуларен и скалабилен системски дизајн кој ги задоволува функционалните и нефункционалните барања на современите софтверски системи. Решението е базирано на примена на слоевита архитектура, при што секој архитектонски слој има јасно дефинирана одговорност и прецизно определена улога во целокупниот систем. Ваквиот архитектонски пристап овозможува зголемена одржливост на системот, олеснето одржување, независен развој на системските компоненти, како и унапредување на безбедносните механизми и оптимизација на перформансите.

Со примената на слоевитата архитектура се овозможува логичко и функционално раздвојување на корисничкиот интерфејс од бизнис логиката и слојот за управување со податоци, како и централизирано управување со процесите за автентикација и контрола на пристап. Дополнително, системот е проектиран со цел да се обезбеди високо ниво на флексибилност, што овозможува негова понатамошна надградба, интеграција со надворешни сервиси и адаптација на нови технолошки парадигми и функционални барања.

Архитектура

Системот е реализиран преку повеќеслојна архитектура со цел да се обезбедат модуларност, скалабилност, безбедност и висок степен на одржливост. Архитектонската структура го вклучува фронтенд слојот (frontend), бекенд слојот (backend), прокси слојот (proxy), системот за управување со идентитет и пристапи (Identity and Access Management,

IAM) и слојот за бази на податоци. Ваквата организација овозможува јасно раздвојување на одговорностите помеѓу различните компоненти, што ја олеснува имплементацијата, тестирањето и одржувањето на системот и обезбедува флексибилност за идни надградби преку додавање на нови слоеви или компоненти, со што се овозможува нивна интеграција во постојната структура без нарушување на функционалноста на системот. Овој пристап значително го подобрува управувањето со комплексноста и го зголемува животниот век на софтверското решение.



Слика 3 Архитектура на апликацијата

Фронтенд слој

Фронтенд слојот го претставува визуелниот и интерактивниот дел на системот, преку кој крајните корисници директно комуницираат со апликацијата. Овој слој е одговорен за прикажување на податоците, примање на кориснички интеракции и пренесување на барањата кон backend слојот. Во рамките на архитектурата се развиени две посебни апликации, прилагодени на различни типови корисници. Првата апликација, **code-help-flutter**, е мобилна апликација изградена со Flutter и е наменета за студенти и ентузијасти кои се заинтересирани за решавање задачи и вежбање програмски концепти. Преку оваа апликација, корисниците можат да се регистрираат, најавуваат, пристапуваат до задачи, учествуваат во форуми и користат дополнителни функционалности на платформата. Втората апликација, **code-help-admin-ui**, е веб апликација развиена со Svelte.js и е наменета за администраторите на системот. Таа овозможува CRUD (Create, Read, Update, Delete) операции врз содржината достапна на крајните корисници, овозможувајќи одржување на

квалитетот и релевантноста на податоците и информациите кои се прикажуваат на платформата. Ваквата поделба овозможува фокусирани интерфејси за различните кориснички улоги и го подобрува корисничкото искуство, истовремено и го олеснува одржувањето и развојот на апликациите.

Proxy слој

Proxy слојот служи како посредник помеѓу клиентските апликации и backend сервисите, играјќи клучна улога во управувањето со сообраќајот, централизацијата на пристапот и обезбедувањето на безбедност. Овој слој овозможува централизирана контрола на пристап, дефинирање на рути и политики, како и заштита на системот од неовластен пристап. Компонентата **code-help-ingress** функционира како HTTP [21] reverse proxy и служи како главна влезна точка за јавниот сообраќај. Таа пренасочува барања кон соодветните backend сервиси или други proxy компоненти, овозможувајќи централизиран и контролирано дефиниран сообраќај. Вториот прокси, **code-help-admin-proxy**, е специјализиран proxy развиен во Go, наменет исклучиво за администраторската апликација. Покрај пренасочување на барањата, овој прокси врши и автентикација и авторизација на корисниците пред пристап до чувствителни ресурси, со што значително ја зголемува безбедноста на административниот дел од системот.

Backend слој

Backend слојот го содржи основниот бизнис логички код и е поделен во два засебни сервиси според доменот на одговорност, со цел да се постигне јасно раздвојување на функционалностите и лесно одржување. Првиот сервис, **code-help-core**, е развиен во Java користејќи ја Spring Boot рамката и е одговорен за основните функционалности на апликацијата, како што се управување со задачи, пресметки на резултати, праќање повратна информација и следење на напредокот на корисниците. Core модулот претставува суштината на платформата и обезбедува стабилна и прецизна имплементација на бизнис логиката. Вториот сервис, **code-help-forum**, е развиен во Kotlin, исто така со Spring Boot, и ги опслужува модулите за форум и дискусии. Тој ги обработува барањата за креирање и одржување на групи, теми, постови и коментари. Форум модулот овозможува размена на знаење и соработка меѓу корисниците, што е од суштинско значење за интерактивноста и заедничкото учење на платформата.

Слој за управување со идентитети, автентикација и авторизација (IAM)

Слојот за управување со идентитети и пристап е задолжен за автентикација и авторизација на сите корисници на системот. Овој слој обезбедува централизирано управување со идентитетите, улогите и дозволите, што е критично за безбедноста и интегритетот на сите останати слоеви на архитектурата. Компонентата **code-help-keycloak-iam** е развиена со интеграција на Keycloak, отворено и робустно решение за управување со идентитети. Keycloak обезбедува централна точка за автентикација и авторизација на мобилната апликација, административната веб апликација и backend сервисите. Повеќе објаснување е наведено во наредните поглавја во врска со безбедноста

Бази на податоци

Слојот за бази на податоци е дизајниран според природата на податоците кои се

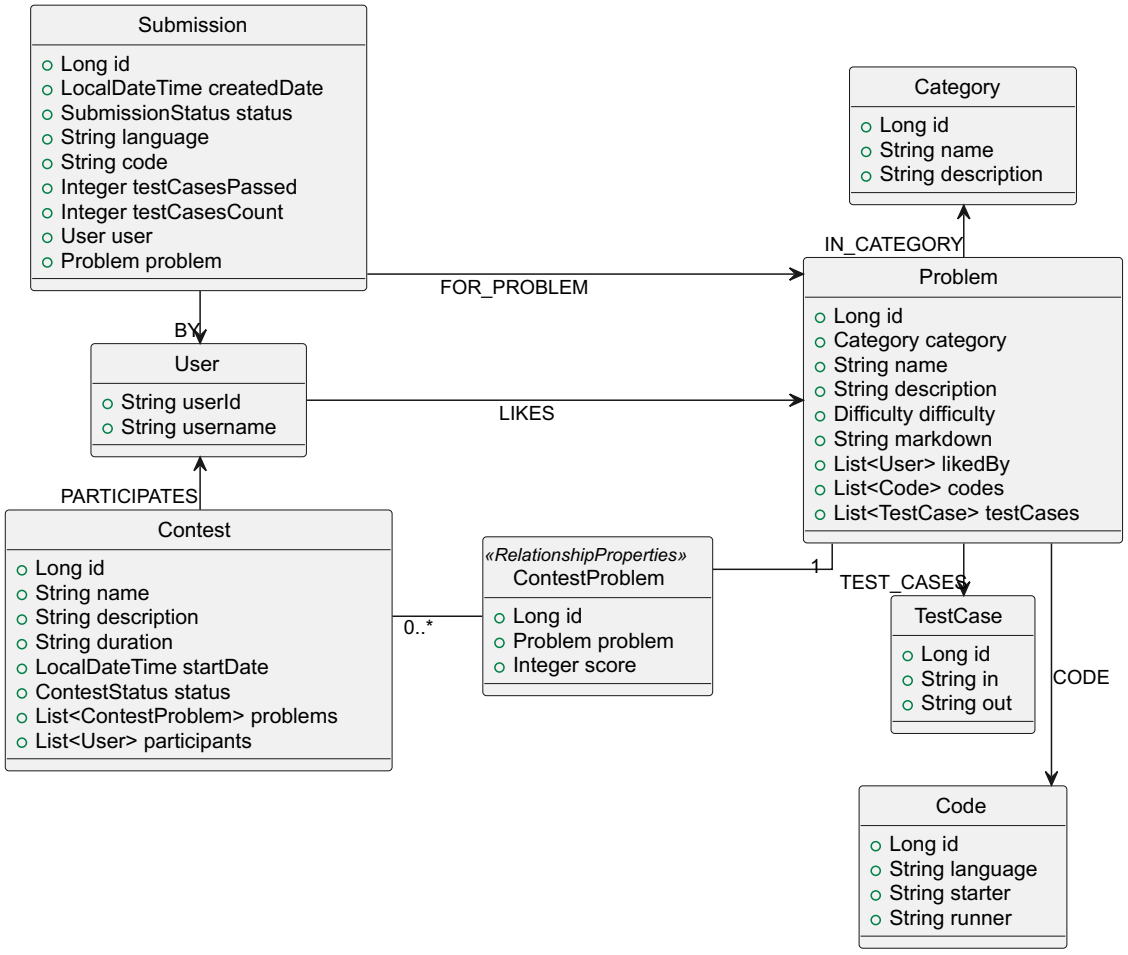
обработуваат и вклучува една граф базирана и релациона база на податоци, што овозможува оптимизирана работа со различни видови информации. Дизајнот на базите на податоци се објаснува во подолните поглавја.

Дизајн на базите на податоци

Дизајнот на базите на податоци на системот е реализиран согласно принципите на моделирање на податоци и е претставен преку Unified Modeling Language (UML) дијаграми, кои обезбедуваат јасна и унифицирана визуелна репрезентација на ентитетите, нивните атрибути и меѓусебните релации. Системот користи хетероген пристап кон управување со податоците, при што за модулот за задачи и натпревари е применета граф база на податоци Neo4j, поради сложените и силно поврзани односи помеѓу корисниците, задачите и натпреварите, додека за форум модулот е користена релациона база на податоци PostgreSQL, која овозможува структурирано складирање, нормализација и одржување на референтниот интегритет на податоците.

Дизајн на базата на податоци за модулот за натпревари и задачи

Прикажаниот модел претставува дизајн на граф база на податоци за модул за програмерски задачи и натпревари, при што податоците се организирани како јазли и релации со експлицитно дефинирани својства. Ваквиот пристап овозможува природно моделирање на комплексните односи помеѓу корисниците, задачите и натпреварите, како и ефикасно извршување на упити кои вклучуваат повеќекратни поврзувања.



Слика 4 Дијаграм на базата на податоци за модулот за решавање на задачи и натпревари

Корисниците во системот се моделирани како јазли од тип **User**, кои содржат основни атрибути за идентификација, како што се единствен идентификатор и корисничко име. Овие јазли се поврзани со други јазли преку различни типови релации, со што се овозможува следење на активностите на корисниците, нивното учество на натпревари и интеракцијата со задачите.

Програмерските задачи се претставени како јазли од тип **Problem**, кои содржат информации за името, описот, тежината и деталната спецификација на задачата. Секоја задача е поврзана со јазол од тип **Category**, со што се обезбедува логичка класификација според тематика. Дополнително, задачите се поврзани со повеќе јазли од тип **TestCase**, кои ги содржат влезните и излезните податоци потребни за автоматска евалуација на решенијата. Ваквата структура овозможува едноставно додавање и модификација на тест случаи без нарушување на останатите делови од моделот.

Поднесените решенија се моделирани како јазли од тип **Submission**, кои содржат податоци за изворниот код, користениот програмски јазик, статусот на извршување и резултатите од тестирањето. Секој јазол од овој тип е поврзан со корисникот кој го поднел решението и со задачата на која се однесува, што овозможува ефикасно следење на историјата на решавање и анализа на успешноста на корисниците.

Поддршката за повеќе програмски јазици и околини за извршување е реализирана преку јазли од тип **Code**, кои содржат информации за почетен код и механизми за извршување. Овие јазли се поврзани со задачите, со што се овозможува флексибилна и проширлива поддршка за различни технологии.

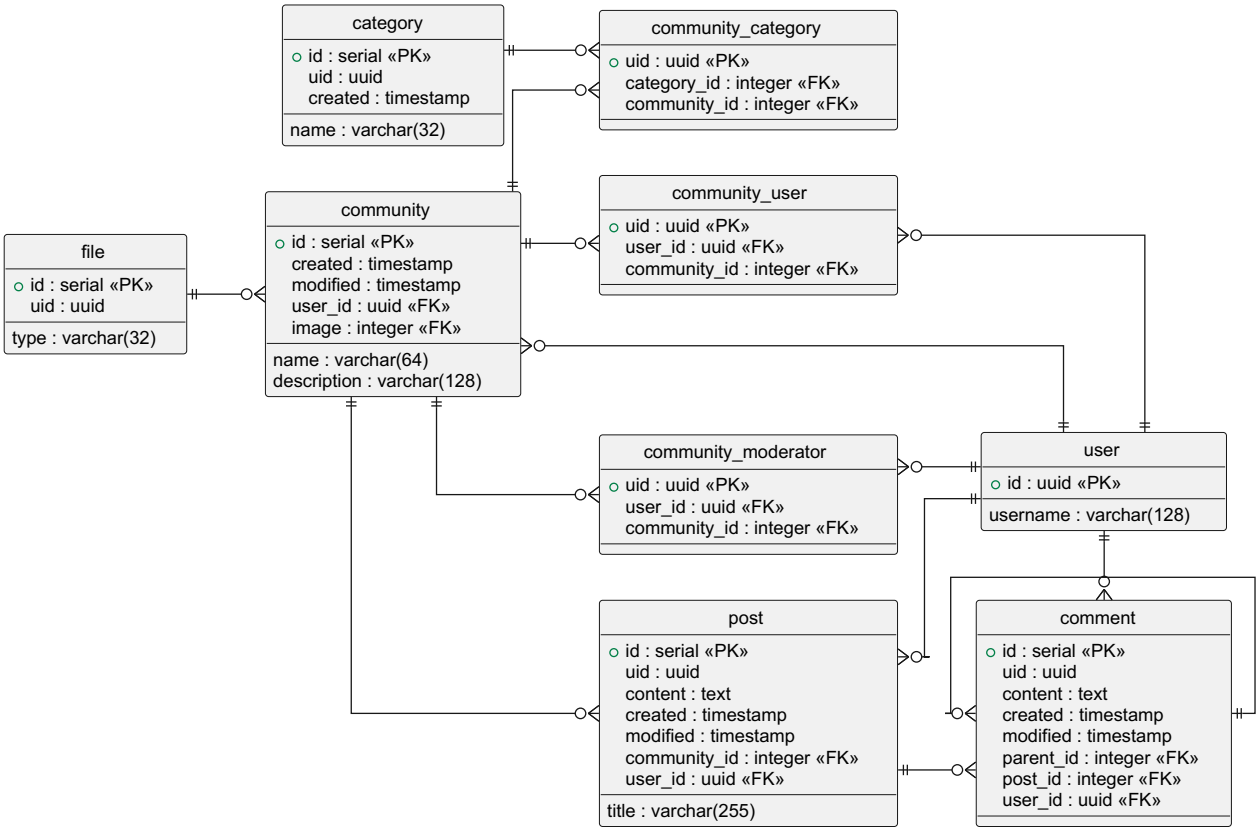
Натпреварите се моделирани како јазли од тип **Contest**, кои содржат информации за името, описот, времетраењето, почетниот датум и статусот. Корисниците се поврзуваат со натпреварите преку релации кои го означуваат нивното учество, додека задачите се поврзуваат преку посредни релации со сопствени својства. Релацијата **ContestProblem** претставува релација со атрибути, при што секоја врска помеѓу натпревар и задача содржи дополнителна информација за бодовната вредност на задачата во рамки на конкретниот натпревар. Ова е типичен пример за предноста на граф базите на податоци, каде што релациите можат да носат сопствени податоци.

Ваквиот граф модел овозможува ефикасно извршување на комплексни упити, како што се добивање на сите задачи решени од одреден корисник во конкретен натпревар, пресметка на бодови или анализа на популарноста на задачите. Со користење на граф база на податоци се намалува сложеноста на упитите во споредба со релациските модели и се подобрува перформансот при работа со силно поврзани податоци и обезбедува флексибилна, скалабилна и семантички јасна структура, соодветна за имплементација на систем за програмерски натпревари и задачи.

Граф ориентираниот пристап овозможува полесно проширување на моделот со нови типови јазли и релации, без значително нарушување на постоечката структура, што го прави соодветен за динамични и еволутивни системи.Дизајн на базата на податоци за форум модулот

Дизајн на базата на податоци за форум модулот

Ова поглавје го опишува дизајнот на базата на податоци за форум модулот, кој е реализиран како релациона база на податоци. Дизајнот е изработен согласно принципите на релациско моделирање, со цел да се обезбеди структурирано, конзистентно и ефикасно управување со податоците поврзани со корисниците, заедниците и содржината генерирана од корисниците.



Слика 5 Дијаграм на базата на податоци за модулот за форум

Во рамки на форум модулот, корисниците се моделирани преку табелата user, која содржи основни атрибути за идентификација, вклучувајќи единствен идентификатор и корисничко име. Корисниците можат да бидат членови на повеќе заедници, што е реализирано преку посредната табела **community_user**. Овој пристап овозможува моделирање на релација од тип повеќе-кон-повеќе, со што се обезбедува флексибилност и јасна евиденција на членството.

Заедниците се моделирани преку табелата community, која содржи информации за името, описот, времето на креирање и последна измена. Секоја заедница е поврзана со корисник кој ја креирал, како и со датотека што претставува визуелна репрезентација, преку странски клуч кон табелата **file**. Овој дизајн овозможува одвојување на мултимедијалните податоци од логичките ентитети, со што се подобрува одржливоста и проширливоста на системот.

Категоризацијата на заедниците е реализирана преку табелата category, која овозможува логичка класификација според тематика. Поврзувањето помеѓу заедниците и категориите е остварено преку табелата **community_category**, што овозможува една заедница да припаѓа на повеќе категории и обратно. Ваквата структура е во согласност со принципите на

нормализација и придонесува за намалување на редундантноста на податоците.

Улогите на корисниците во рамки на заедниците се моделирани преку табелата **community_moderator**, која овозможува издвојување на корисници со модераторски привилегии. Со ова се обезбедува јасна контрола на пристапот и управувањето со содржината во рамки на форумот, без нарушување на основниот модел на членство.

Содржината креирана од корисниците е организирана преку табелата **post**, која ги моделира објавите во рамки на заедниците. Секоја објава содржи наслов, текстуална содржина и временски ознаки за креирање и измена, а е поврзана со заедницата во која е објавена и со корисникот кој ја креирал. Овој дизајн овозможува јасна поврзаност помеѓу корисниците и објавената содржина.

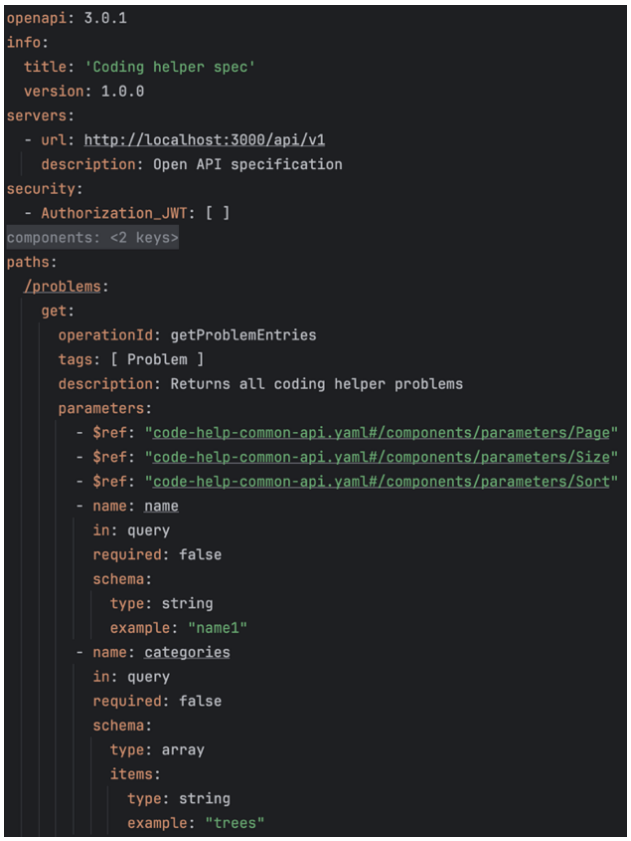
Коментарите се моделирани преку табелата **comment**, која овозможува дискусија во рамки на објавите. Секој коментар е поврзан со конкретна објава и корисник, а содржи информации за содржината и временските ознаки. Ваквата структура овозможува хиерархиско организирање на содржината и следење на интеракцијата помеѓу корисниците.

Претставениот дизајн на релационата база на податоци обезбедува нормализирана, логички кохерентна и проширлива структура за форум модулот и поддржува управување со заедници, корисничка интеракција и креирање на содржина, изградена согласно со современите принципи на дизајн на релациони бази на податоци.

Дизајнирање на Апликациски програмски интерфејс со репрезентативен пренос на состојба

Апликациските програмски интерфејси (API) се составен дел од современиот развој на софтвер, овозможувајќи им на различни системи непречено да комуницираат и разменуваат податоци меѓусебно. Како што побарувачката за меѓусебно поврзани и дистрибуирани системи продолжува да расте, дизајнот на овие апликациски програмски интерфејси станува се поважен. Добро дизајнираниот апликациски програмски интерфејс може значително да ја подобри продуктивноста на развивачите како и скалабилноста на апликациите и целокупната одржливост на системот.

Во оваа апликација се користат најдобрите практики за дизајнирање на едно REST API со цел лесно управување со системот. Основните принципи на REST се следени за да се обезбеди чиста и лесно одржлива архитектура.



```yaml
openapi: 3.0.1
info:
  title: 'Coding helper spec'
  version: 1.0.0
servers:
  - url: http://localhost:3000/api/v1
    description: Open API specification
security:
  - Authorization_JWT: [ ]
components: <2 keys>
paths:
  /problems:
    get:
      operationId: getProblemEntries
      tags: [ Problem ]
      description: Returns all coding helper problems
      parameters:
        - $ref: "code-help-common-api.yaml#/components/parameters/Page"
        - $ref: "code-help-common-api.yaml#/components/parameters/Size"
        - $ref: "code-help-common-api.yaml#/components/parameters/Sort"
        - name: name
          in: query
          required: false
          schema:
            type: string
            example: "name1"
        - name: categories
          in: query
          required: false
          schema:
            type: array
            items:
              type: string
              example: "trees"
```

Слика 6 OpenAPI конфигурација во YAML формат

Апликацијата имплементира архитектура заснована на ресурси, при што за интеракција со ресурсите се користат стандардни HTTP методи како GET, PUT, PATCH, POST и DELETE. [7] Овие методи овозможуваат операции како земање податоци, ажурирање, парцијално ажурирање, креирање и бришење на ресурси. За полесна и поефикасна интеграција со други системи, дефинирани се посебни OpenAPI конфигурации во YAML формат во датотеки за модулите во апликацијата. Тие YAML датотеки ја дефинираат целосната структура на апликацијата и прецизираат дали е потребно да се прати безбедносен токен при повикување на некоја рута или создавање HTTP барање. Исто така, се одредува што треба или што не треба да се вклучи во барањето, кои параметри се задолжителни, кои рути се достапни и други важни детали. На сликата е прикажан пример за структурата на овие датотеки.

JSON и HAL одговор (application/json+hal)

За размена на податоци помеѓу клиентот и серверот, како и за интероперабилност меѓу различни сервиси, се користи широко распространетиот JSON (JavaScript Object Notation) формат. [22] JSON е структурирана, текстуална форма на податоци која е лесна за читање и пишување, овозможувајќи ефикасен и прецизен пренос на информации помеѓу различни платформи и програмски јазици. Неговата едноставна и универзална структура ја олеснува обработката на податоците на клиентската и серверската страна, овозможувајќи брза серијализација и десеријализација на комплексни податоци, како и интеграција со различни апликациски слоеви во рамките на современи веб и мобилни архитектури.

Хипертекст апликацискиот јазик (HAL) [23] претставува стандардизирано проширување на JSON за RESTful апликации, кое додава хипер-врски за претставување на ресурси и нивните меѓусебни релации. HAL овозможува клиентските апликации да ги користат овие врски за навигација низ ресурсите на серверот без потреба од познавање на целокупната структура на API-то или дополнителни конфигурации. Овој пристап овозможува автоматско откривање на поврзани ресурси, поедноставена интеграција со други сервиси и значително ја зголемува автономијата на клиентските апликации. На овој начин се создава флексибилна, скалабилна и сигурна комуникациска архитектура, која поддржува динамичко користење на податоците и олеснува одржување и проширување на системот.


```
{
  "totalElements": 5,
  "totalPages": 3,
  "pageSize": 2,
  "pageNumber": 1,
  "links": {
    "self": {
      "href": "http://localhost:3000/api/v1/problems?page=1&size=2"
    },
    "first": {
      "href": "http://localhost:3000/api/v1/problems?page=0&size=2"
    },
    "last": {
      "href": "http://localhost:3000/api/v1/problems?page=3&size=2"
    },
    "prev": {
      "href": "http://localhost:3000/api/v1/problems?page=0&size=2"
    },
    "next": {
      "href": "http://localhost:3000/api/v1/problems?page=2&size=2"
    }
  },
  "problems": [
    {
```


Слика 7 JSON и HAL одговор

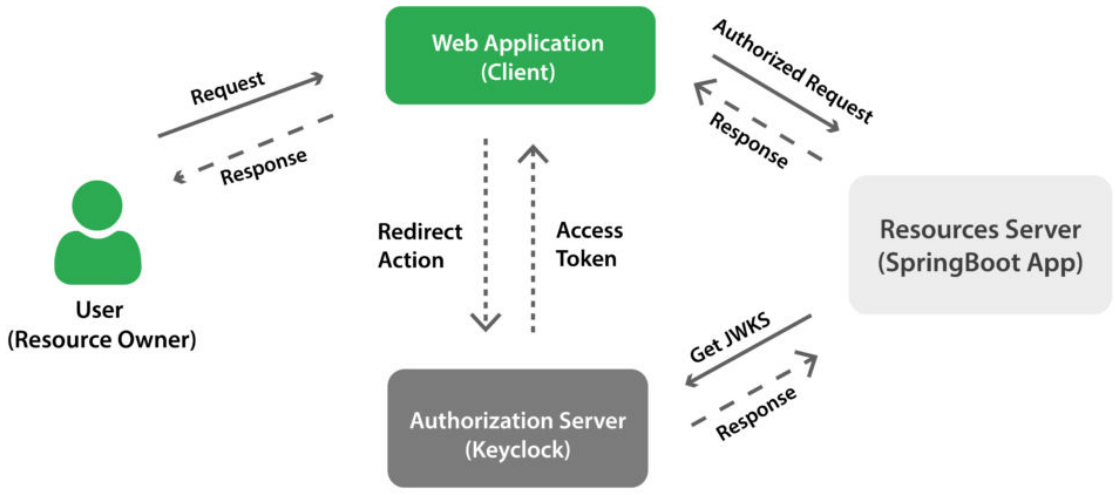
На сликата е даден пример со пагинација каде што објектот се проширува со links објект, при што со ова проширување, на клиентите им се овозможува полесно управување со податоците.

Безбедност во апликациите

Безбедноста на апликациите и податоците претставува еден од најважните аспекти во денешниот дигитален свет. Со зголемената употреба на интернет услуги и апликации, како и со постојаното зголемување на сајбер заканите, неопходно е да се имплементираат сигурни механизми за автентикација и авторизација. Овие механизми не само што ја штитат доверливоста на податоците, туку и обезбедуваат сигурност на целокупниот систем.

Еден од најефикасните начини за постигнување на ова е употребата на алатки за управување со идентитети и безбедносни токени, како што е Keycloak. Keycloak се користи како Провајдер на идентитети (IdP) и го управува процесот на автентикација и авторизација на корисниците преку улоги и безбедносни политики, како централен орган за сигурност на апликациите. Keycloak е базиран на стандардизирани протоколи како OAuth 2.0 за авторизација и OpenID Connect за автентикација, овозможувајќи генерација на сигурни токени кои ги дефинираат привилегиите на корисниците.

Во предложената архитектура, Spring Boot [1] апликацијата функционира како сервер за ресурси, кој ја обработува бизнис логиката и управува со податоците. Таа ги верификува токените добиени од Keycloak за да осигура безбедна комуникација и контролиран пристап до ресурсите. На овој начин се воспоставува јасна поделба на одговорностите: Keycloak ја извршува автентикацијата и управува со идентитетите, додека Spring Boot ја реализира авторизацијата и заштитата на податоците на ниво на апликација.



Слика 8 Автентикација и авторизација со Keycloak и Spring Boot

Овој процес се одвива во четири чекори: иницијална автентикација на корисникот, издавање на токен, испраќање на токенот при пристап до заштитени ресурси, и валидација на токенот за обезбедување на контролирана авторизација.

Целокупниот систем директно ја поддржува CIA тријадата:

- **Доверливост**: Контролираниот пристап преку Keycloak и улогите во Spring Boot овозможува

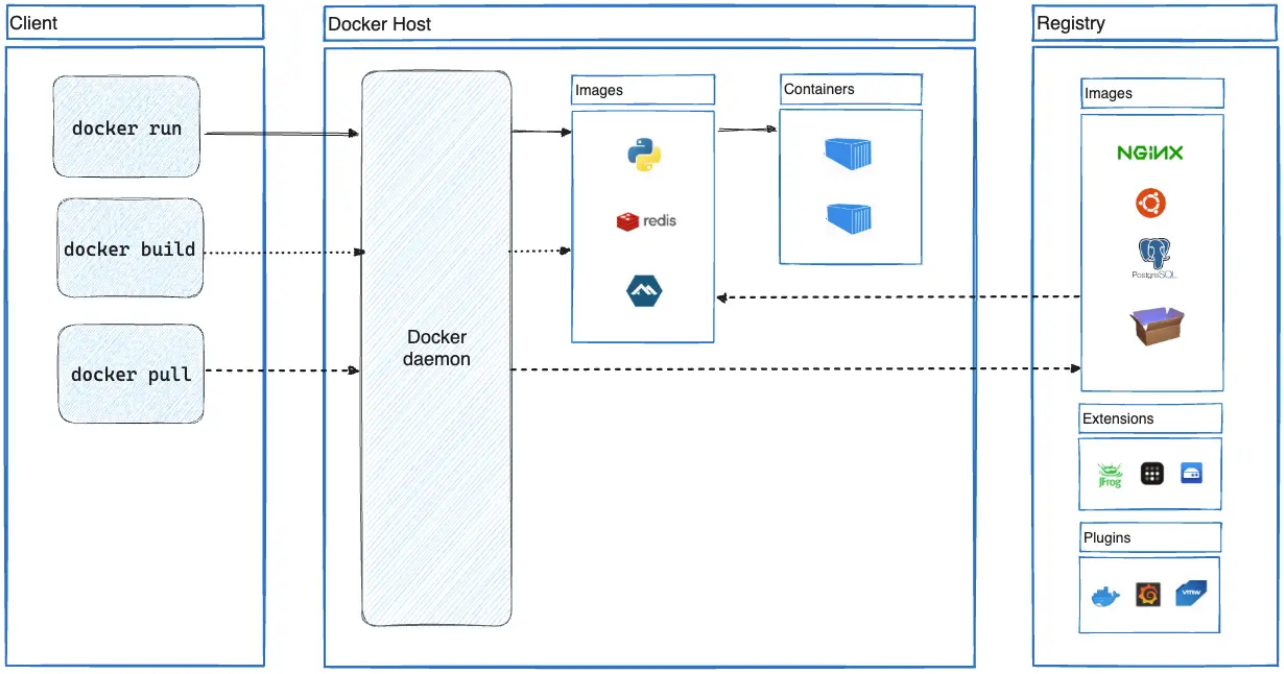
дека податоците се достапни само за овластени корисници. HTTPS и токените дополнително го заштитуваат преносот на чувствителни информации.

- **Интегритет**: JWT токените се дигитално потпишани и се верификуваат од Spring Boot, што гарантира дека податоците и информациите за корисникот не се менуваат на неовластен начин.
- **Достапност**: Децентрализираниот и скалабилен дизајн на Keycloak и Spring Boot овозможува висока достапност на услугите дури и при зголемено оптоварување или делумни проблеми.

Имплементацијата на овие механизми обезбедува ефикасен, сигурен и скалабилен систем за автентикација и авторизација, кој ги штити податоците и услугите од најчестите сајбер закани.

Контејнеризација и градење на апликацијата

Контејнеризацијата претставува современ пристап за извршување на софтверски апликации, при што апликацијата се пакува заедно со сите нејзини зависности во изолирана околина. Овој пристап овозможува апликацијата да се извршува на ист начин во различни средини, без разлика дали станува збор за развојна, тест или продукциска околина. Со тоа се намалуваат проблемите поврзани со различни конфигурации на системите и се зголемува стабилноста на апликацијата.



Слика 9 Архитектура на Docker

За разлика од традиционалната виртуелизација, каде секоја виртуелна машина содржи сопствен оперативен систем, контејнерите го користат јадрото на хост оперативниот систем. Ова резултира со значително помала потрошувачка на ресурси, побрзо стартување и поефикасно искористување на хардверот, што ги прави контејнерите особено погодни за современи, дистрибуирани апликации.

Една од пошироко познатите алатки која се користи за контејнеризација на софтверско решение и креирање на слики од компонентите е алатката Docker кој е отворена платформа

за развој, испорака и извршување на апликации. Docker е дизајниран врз основа на клиент-сервер архитектура, кој овозможува јасна функционална поделба помеѓу компонентите задолжени за управување и оние задолжени за извршување на апликациите. Интеракцијата со Docker се остварува преку Docker клиентот кој е служи како интерфејс за испраќање команди и барања кон системот. Овие барања се проследуваат до Docker daemon преку дефиниран програмски интерфејс, со што се обезбедува централизирано и контролирано управување со процесите поврзани со контејнеризацијата како градење, извршување и дистрибуирање на контејнерите.

Docker daemon функционира како сервис што работи во позадина на оперативниот систем и е одговорен за управување со животниот циклус на Docker слики и контејнери. Неговите надлежности вклучуваат градење на слики, стартување и стопирање на контејнери, како и управување со комуникацијата помеѓу контејнерите и останатите системски ресурси.

Во процесот на градење на апликацијата, Docker daemon може да користи постоечки слики преземени од регистер на слики или да креира нови слики врз основа на инструкции дефинирани во Dockerfile.

```
1  FROM openjdk:17-slim
2
3  ARG VERSION='0.0.1-SNAPSHOT'
4  ENV JAR_PATH code-help-core-app/target/code-help-core-app-$VERSION.jar
5  ENV PROFILE docker
6  ENV JAVA_OPTIONS ''
7  ENV JAVA_PROPERTIES ''
8
9  WORKDIR /code-help
10
11 RUN apt update -y && \
12     apt upgrade -y && \
13     apt install -y nodejs
14
15 COPY $JAR_PATH core-app.jar
16
17 EXPOSE 3000
18
19 ENTRYPOINT exec java $JAVA_OPTIONS -jar \
20               -Dspring.profiles.active="$PROFILE" \
21               $JAVA_PROPERTIES core-app.jar
```

Според примерот од сликата претставен е еден **Dockerfile** кој овозможува да се креира Docker слика која ќе ја користи веќе креираната сликата преземена од Docker регистарот на слики наречена **openjdk** која е со верзија **17-slim**, а потоа се ажурираат веќе инсталираните пакети и се инсталираат потребните пакети од кој зависи овој контејнер внатре во него и се прави копија од **JAR** датотеката така што се копира внатре во контејнерот. За крај **ENTRYPOINT** командата кажува што ќе се случи кога оваа слика ќе се искористи со **docker run** командата.

Compose датотека

Docker Compose датотеката претставува конфигурација базирана на YAML синтакса,

преку која се дефинираат и конфигурираат сите апликациски сервиси потребни за функционирање на системот. Преку оваа датотека се централизира конфигурацијата на повеќеслојната апликација и се овозможува нејзино едноставно стартување и управување. Користењето на YAML конфигурација овозможува јасно и структурирано опишување на сервисите и нивните меѓусебни зависности, со што се обезбедува конзистентно извршување на апликацијата во различни околини. Ако го разгледаме прикажаниот дел од датотеката ќе видиме дека аргументите и варијаблите кои ни се потребни ги дефинираме во оваа датотека.

```
version: '3'
services:
  code-help-core:
    image: code-help-core
    container_name: code-help-core
    build:
      context: code-help-core
      args:
        - VERSION=0.0.1-SNAPSHOT
    environment:
      SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUERURI: http://10.0.2.2/iam/realms/code-help
      SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWKSETURI: http://code-help-keycloak-iam:8080/iam/realms/code-help/protocol/openid-connect/certs
      SPRING_NEO4J_URI: bolt://code-help-core-neo4j:7687
      SPRING_NEO4J_AUTHENTICATION_USERNAME: neo4j
      SPRING_NEO4J_AUTHENTICATION_PASSWORD: password
    ports:
      - 3000:8080
    depends_on:
      - code-help-core-neo4j
  code-help-core-neo4j:
    image: neo4j:5.2.0
    hostname: neo4j
    container_name: code-help-core-neo4j
    ports:
      - 7474:7474
      - 7687:7687
    environment:
      - NEO4J_AUTH=neo4j/password
    volumes:
      - code-help-core-neo4j-data:/data
      - code-help-core-neo4j-plugins:/plugins
      - code-help-core-neo4j-logs:/logs
  ....
volumes:
  code-help-core-neo4j-data:
  code-help-core-neo4j-logs:
  code-help-core-neo4j-plugins:
  ....
```

Со извршување на командата **docker-compose -f docker-compose-dev.yml up -d** се иницира креирање на потребната Docker мрежа и стартување на сите дефинирани апликациски сервиси во позадина. Секој сервис е стартуван во посебен Docker контејнер, што овозможува изолирано и контролирано извршување на компонентите на системот.

Успешното стартување на контејнерите укажува дека апликациските сервиси, вклучувајќи ги

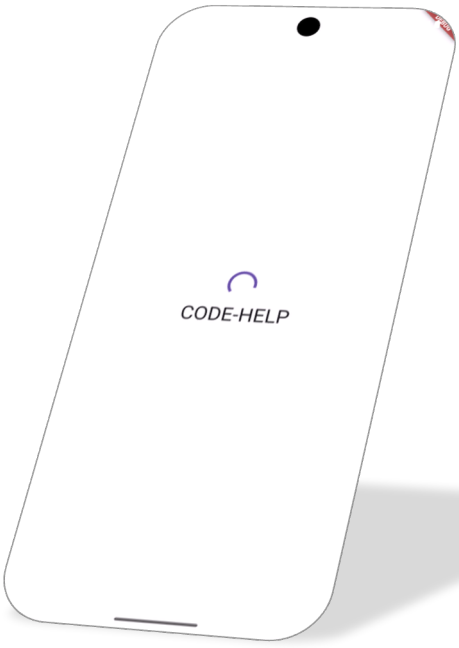
корисничкиот интерфејс, базите на податоци и останатите поддржувачки компоненти, се правилно конфигурирани и подготвени за извршување во дефинираната развојна околина.

```
[mtrajkov@mb5469 coding-helper-cross-platform % docker-compose -f docker-compose-dev.yml up -d
 WARN[0000] /Users/mtrajkov/Desktop/coding-helper-cross-platform/docker-compose-dev.yml: the at
 [+] Running 9/9
  ✔ Network coding-helper-cross-platform_default  Created
  ✔ Container code-help-keycloak-iam              Started
  ✔ Container code-help-admin-ui                  Started
  ✔ Container code-help-forum-postgres            Started
  ✔ Container code-help-core-neo4j                Started
  ✔ Container code-help-core                      Started
  ✔ Container code-help-admin-proxy               Started
  ✔ Container code-help-forum                     Started
  ✔ Container code-help-ingress                   Started
```

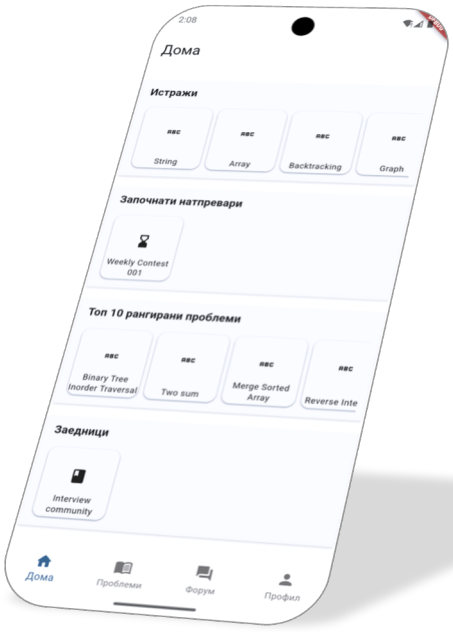
Слика 10 Градење и извршување на апликацијата

Преглед на апликација

Според до сега наведеното заедно со функционалностите на апликацијата, во ова поглавје ќе биди опишано како се користи апликацијата. Кога ќе се отвори апликацијата прво што се појавува на екранот на корисникот е почетниот екран. На почетниот екран се прикажува името на апликацијата и еден спинер кој врти. Тоа му укажува на корисникот дека нешто во позадина се процесира или се извршува некоја работа. Во тој момент апликацијата се обидува да го најави корисникот, ако веќе постои и бил најавен претходно, и го иницира контекстот на апликацијата. По иницирање на апликацијата, се појавува Дома табот независно дали корисникот бил најавен или не, кој ги покажува најбитните делови како прв поглед кон апликацијата.



Слика 11 Почеток на апликација



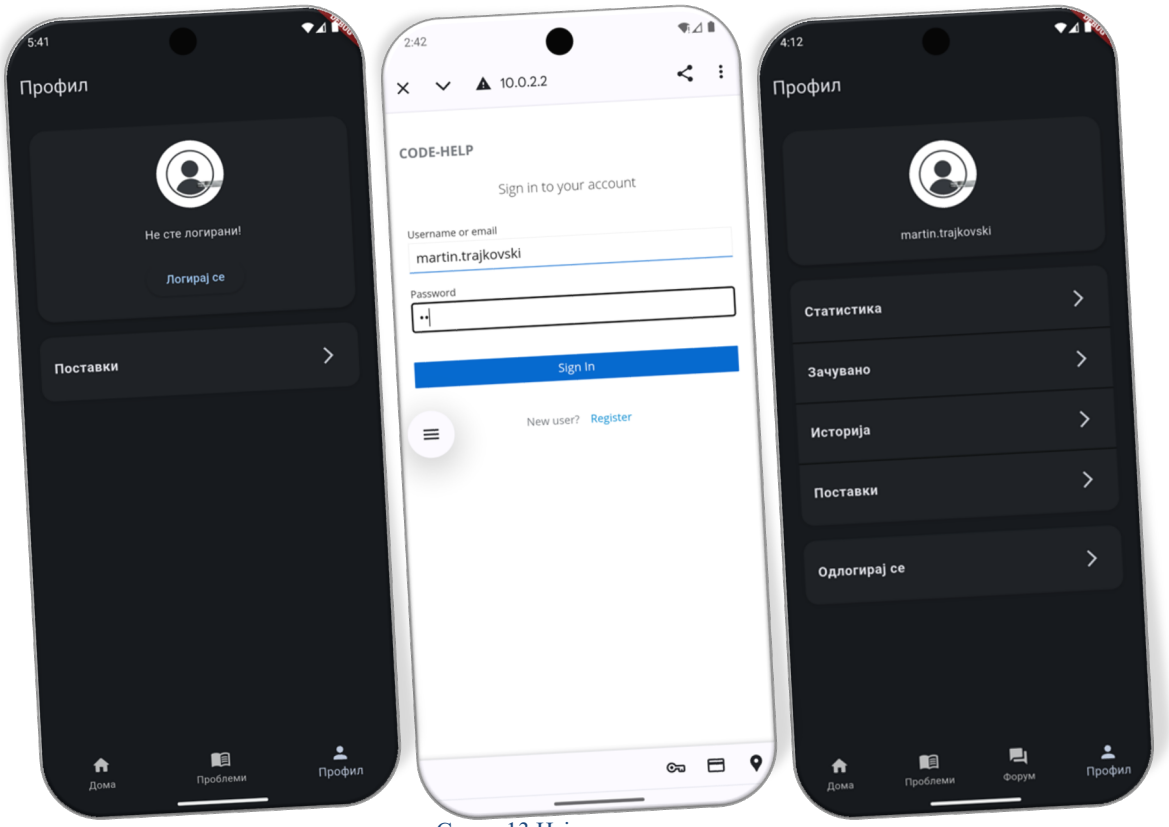
Слика 12 Почетна страница

Најава или регистрација на корисник

Корисникот кога ќе ја отвори апликацијата, можи да се најави или регистрира преку Профил табот на долното мени за навигирање. Кога ќе го отвори Профил табот се појавува

екран со копче за поставки, информација која му укажува на корисникот дека сеуште не е најавен во апликацијата, и копче за најава. При клик на копчето „Логирај се”, корисникот се пренасочува кон веб пребарувач кој го носи корисникот до страната преку која можи да се најави или регистрира. За најава во апликацијата се користи корисничко име или електронска пошта, и лозинка. Ако корисникот внеси грешни податоци, тогаш на екранот ќе се појави информација дека внесените податоци се погрешни, што му укажува на корисникот дека треба повторно да ги внеси точните податоци за најавување. Кога корисникот успешно ќе ги внеси податоците за најава, тогаш тој ќе биде пренасочен назад кон апликацијата.

Ако корисникот прв пат пристапува до апликацијата, тогаш тој ќе мора прво да се регистрира со клик на копчето „Register”, да креира кориснички профил. При клик на копчето за регистрација, корисникот се пренасочува кон друга страна преку која може да ги внеси потребните информации за еден кориснички профил. Тоа се корисничко име, електронска пошта, име, презиме и лозинка. Откако ќе ги внеси овие податоци и внесените податоци се точни и креирањето на профилот е успешно, тогаш тој автоматски ќе биде најавен во апликацијата, при што се доаѓа до третиот екран од апликацијата.



Слика 13 Најава на корисник

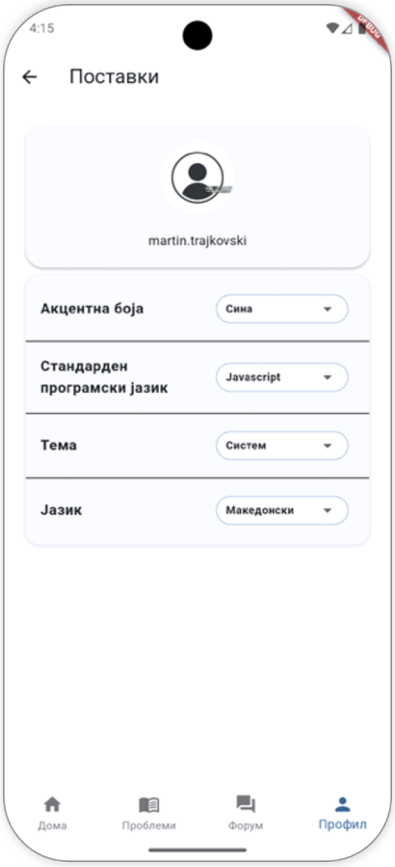
Поставки во апликацијата

Корисникот може да управува со неколку поставки во апликацијата и тоа се: Акцентна боја, Стандарден програмски јазик, Тема и Јазик. Со овие поставки на корисникот му се овозможува подобро корисничко искуство при користење на апликацијата. Сите овие опции се зачувуваат и при рестартирање и повторно приклучување на апликацијата.
Со промена на Акцентната боја, се менува изгледот на целата апликација, достапни бои за селекција се сина, виолетова и црвена. Со оваа поставка корисникот може да си ја избери бојата и изгледот кој најмногу му се допаѓа.

Опцијата за Стандарден програмски јазик се користи за тоа кога корисникот ќе отвори некоја задача што би сакал да ја решава и ако таа задача има можност да се реши во тој програмски јазик односно постои стартирачки код зачуван во базата податоци кој може да се врати од страна на ресурсниот сервис, тогаш тој програмски јазик ќе виде автоматски избран од страна на апликацијата. Во случај корисникот да сака да ја решава таа задача во друг програмски јазик тогаш тој ќе треба да го менува рачно или пак да постави друг стандарден програмски јазик.

Опцијата тема се користи за селектирање на темата на целата апликација. Вредностите кои може да се користат се Светла, Темна и Систем. Вредноста Систем овозможува земање на стандардната тема од системот на уредот што корисникот веќе ја поставил при конфигурација на самиот уредот.

Опцијата Јазик се користи за промена на јазикот во апликацијата. Оваа апликација овозможува селекција на три јазици, и тие се: Македонски, Англиски и Германски. Со оваа опција корисникот може да го користи неговиот мајчин јазик или друг јазик ако тој е достапен во листата на вредности.



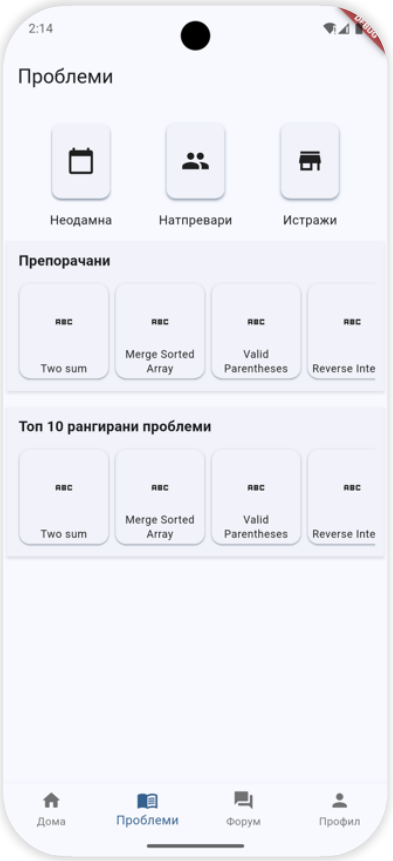
Слика 14 Поставки

Задачи (Проблеми)

Во табот за проблеми се прикажуваат препорачаните задачи, како и топ 10-те највисоко рангирани проблеми во апликацијата. Од оваа страна, корисникот има можност да избере задача од листата на прикажани проблеми и да пристапи до нејзините детали.

За добивање на податоците се извршува повик кон серверот, кој враќа информации за задачите во минимален формат, доволен за прикажување на содржината и основните атрибути на секоја задача. Од оваа точка, корисникот може да навигира кон натпреварите и да се приклучи во конкурентен модул со други програмери, да избере играчи или да истражува и пребарува други проблеми според нивната категорија и природа.

Во наредните поглавја ќе бидат опишани дополнителните прегледи и функционалности кои се достапни од оваа страна, со цел да се обезбеди целосна слика за интеракцијата на корисникот со платформата.



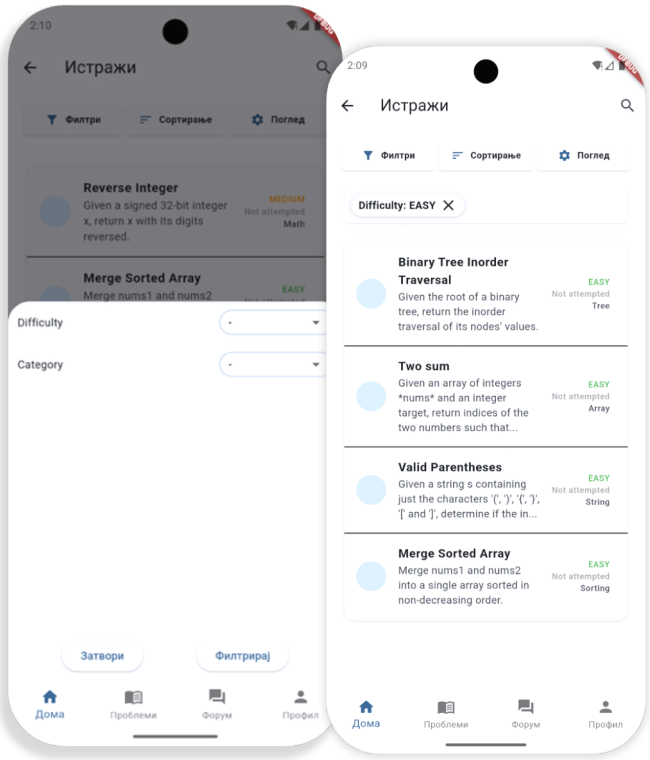
Слика 15 Проблеми насловна

Филтрирање, сортирање, поглед

На истражи екранот корисникот може да ги филтрира веќе постоечките задачи, ги сортира или да го менува погледот на редицата.

Опциите Филтри и Сортирање имаат слична акција, така што при клик на кое било од двете, се појавува модал на екранот, каде што корисникот ќе може да селектира опција по која ќе филтрира или сортира задачи. Откако ќе ги селектира опциите со притискање на копчето Филтрирај или Сортирај, се праќа барање до сервисот со што се враќаат бараните податоци.

Како и во табот проблеми исто така и тука се зема минимален облик од една задача за да не се оптоварува серверот со големи податоци.

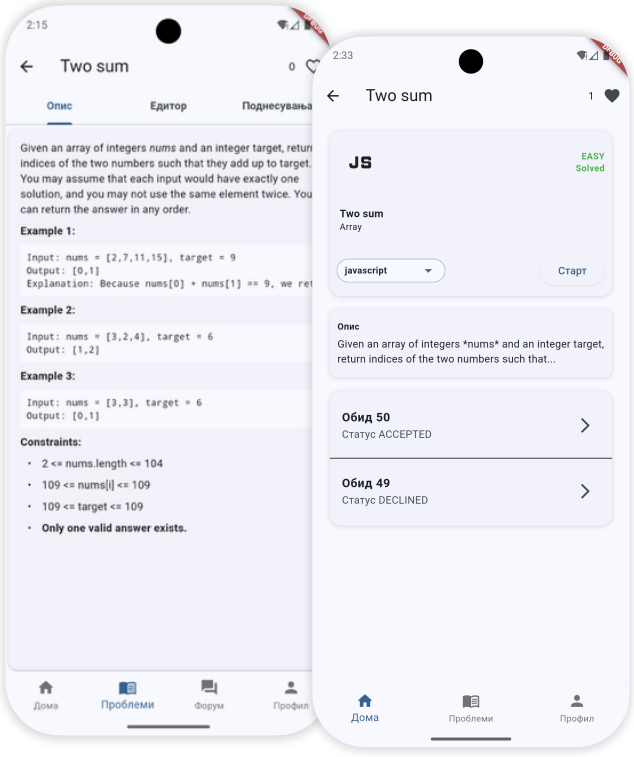


Слика 16 Истражување и филтрирање на задачи

Арена

Кога корисникот ќе отвори некоја задача, при клик на копчето „Старт”, ќе биде пренасочен кон арена екранот каде што ќе можи да почни со решавање на задачата во претходно избраниот програмски јазик. На екранот се појавуваат три табови и тоа Опис, Едитор и Поднесувања.

Во табот Опис, се прикажува описот на задачата што моментаолно е отворена од страна на корисникот која би сакал да ја решава. Описот се состои од тоа што треба да се направи за да се реши задачата, прикажани се неколку тест примери за да корисникот има подетален увид во тоа што треба да врати како резултат од оваа задача и ограничувањата на тест примерите во кој треба да работи решението на корисникот.

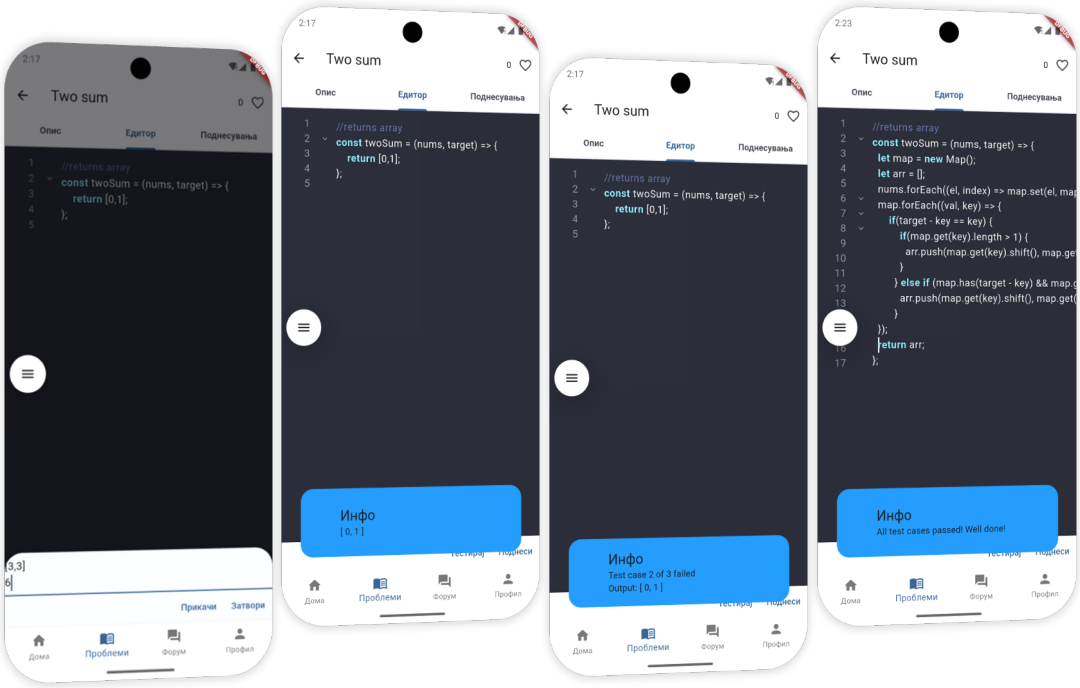


Слика 17 Задача и опис

Во табот Едитор, се прикажува едитор за код, тука корисникот може да го пишува решението на задачата, да го тестира неговиот код преку свој тест примери и да го поднеси решението. При поднесување на решението тоа се тестира со сите тест примери за таа задача. Ако решението на задачата е валидно тогаш се праќа порака на корисникот дека се

поминати сите тест примери и решението е земено за точно, во спротивно се праќа порака на кој тест пример решението е невалидно.

Во табот Поднесувања, се прикажани сите поднесувања на корисникот за задачата со историја на кодот и кога тоа решение било поднесено.

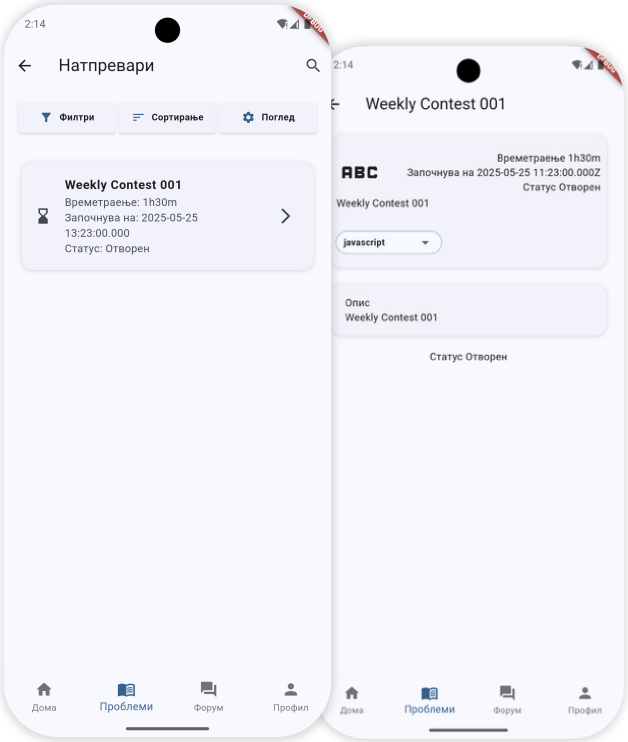


Слика 18 Поднесување и тестирање на решение

Натпревари

Во табот за натпревари, функциите за навигација, филтрирање и сортирање се имплементирани на сличен начин како кај интерфејсот за задачи, со цел да се одржи конзистентност во корисничкото искуство.

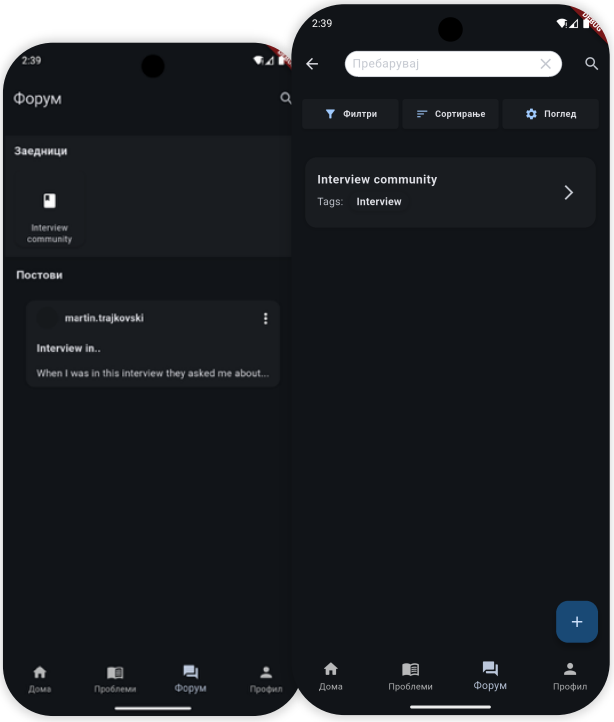
За секој натпревар се прикажуваат релевантни информации, вклучувајќи го времетраењето на натпреварот, точниот почеток и тековниот статус на натпреварот. По започнување на натпреварот, корисникот добива пристап до задачите кои може да ги решава во рамките на дефинираното времетраење на натпреварот. Секое решение што корисникот го прикачува се евалуира автоматски, при што се пресметуваат освоените поени за задачата врз основа на бројот на тестови кои решението ги поминало успешно. Овој пристап овозможува реално и објективно мерење на перформансите на учесниците во текот на натпреварот.



Слика 19 Натпревари и преглед на натпревар

Форум

На почетниот екран во табот за форум се прикажуваат неколку заедници, како и најновите објави од корисниците на апликацијата. Функционалностите кои служат за филтрирање и сортирање се дизајнирани да бидат конзистентни со другите екрани, овозможувајќи корисниците лесно да ги пронајдат релевантните содржини. Корисникот има можност да креира нова заедница или да се приклучи на веќе постоечките, да објавува свои постови, да коментира на содржини од други корисници и да учествува во интеракции со членовите на заедницата.

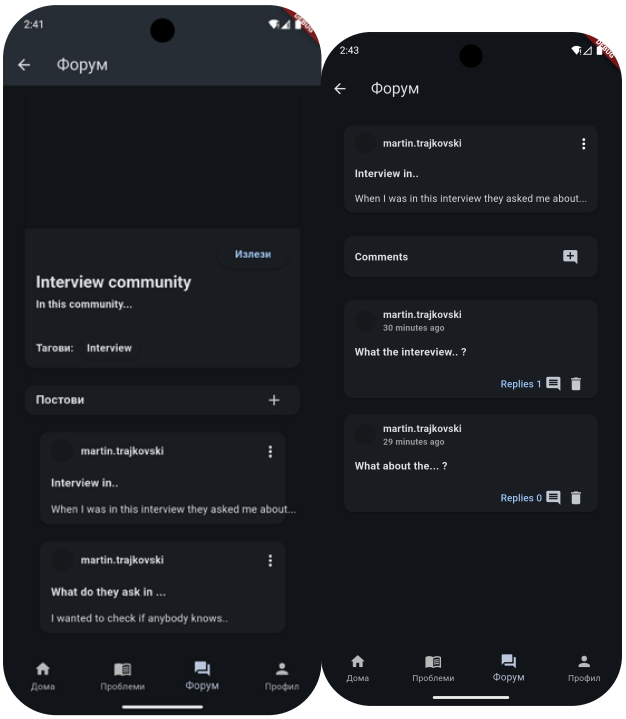


Слика 20 Форум почетна страна и пребарување на заедници

Овој пристап кон дизајнот на форумот е инспириран од онлајн платформата Reddit[2], прилагоден за потребите на апликацијата, со цел да се овозможи лесно користење, структурирана организација на заедниците и поттикнување на интеракција и размена на знаење меѓу корисниците.

Заедница

На екранот на една заедница се прикажуваат клучните информации за избраната заедница, вклучувајќи опис, членови и активни постови. Корисникот може да ги прегледува објавите сортирани по најнови, како и да креира нови постови, да коментира на постовите на други корисници и да одговара на коментари, со што се овозможува активна интеракција во рамките на заедницата.



Слика 21 Заедница и постови со коментари

За да се приклучи како член на заедницата, корисникот користи копчето „Влези“. Откако ќе стане член, ова копче се менува во „Излези“, кое овозможува лесно напуштање на заедницата со еден клик. Секој корисник има можност не само да постира и коментира, туку и да уредува или брише свои објави и коментари, обезбедувајќи целосна контрола над сопствената содржина и интеракции во заедницата.

[2] Reddit е едно решение на форум платформа https://reddit.com

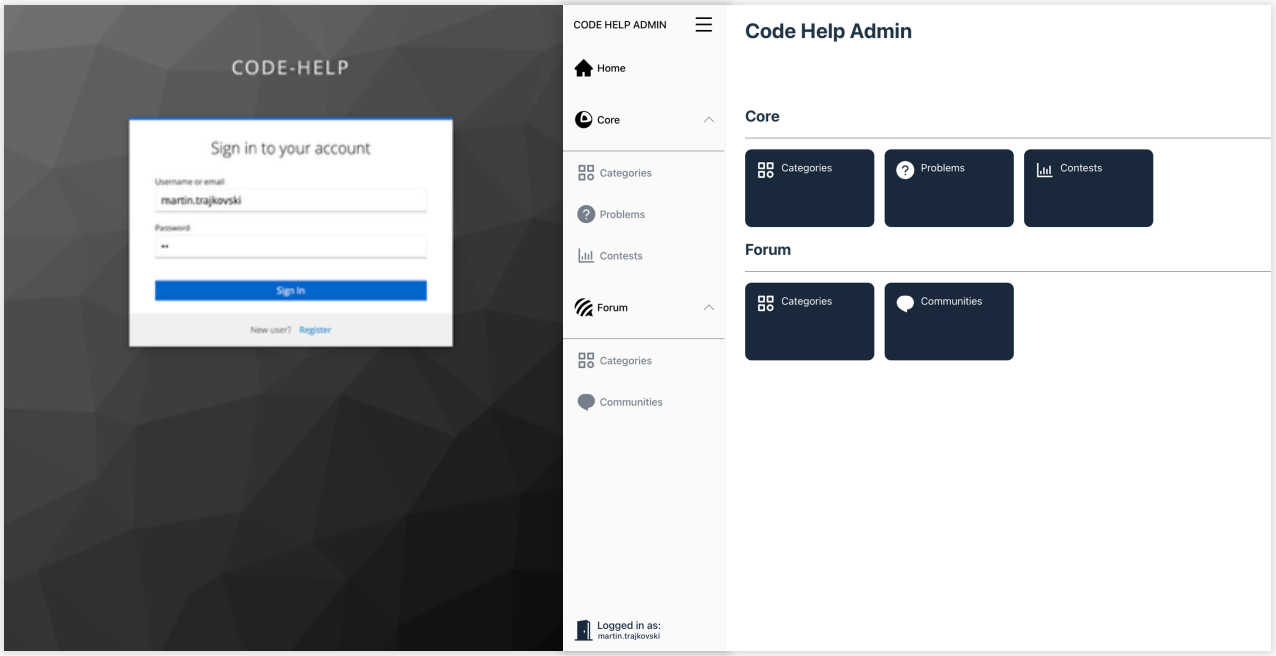
Користење на Админ апликација

Администраторскиот дел каде што еден админ корисник може да управува со податоците во целокупниот систем се состои од посебна администраторска апликација со кориснички интерфејс кој го олеснува управувањето со податоци за модулите.

Процесот започнува со најава на корисникот преку форма за автентикација, при што корисникот ги внесува своите кориснички креденцијали (корисничко име и лозинка). Автентикацијата се извршува преку надворешен сервис за управување со идентитети и пристап, кој е одговорен за проверка на валидноста на податоците и за доделување на соодветни привилегии. Само корисници со администраторска улога добиваат дозвола за пристап до админ апликацијата.

По успешна најава, корисникот се пренасочува кон почетната страна на администраторскиот панел. Како што е прикажано на сликата, интерфејсот е организиран со странично навигациско мени, преку кое администраторот може да пристапи до различни функционални модули на системот. Овие модули се групирани во логички целини, како што се управување со податоците за модулот за решавање на задачи и натпревари или јадрен модул и модулот за управување со форумски содржини.

Во рамките на секој модул, администраторот има можност да управува со ентитети како што се категории, проблеми и придонеси. Системот овозможува преглед на постоечките записи, нивно пребарување, ажурирање и бришење, како и додавање на нови податоци. Овој пристап овозможува ефикасно и централизирано администрирање на содржината, како и подобра контрола врз податоците во базата.

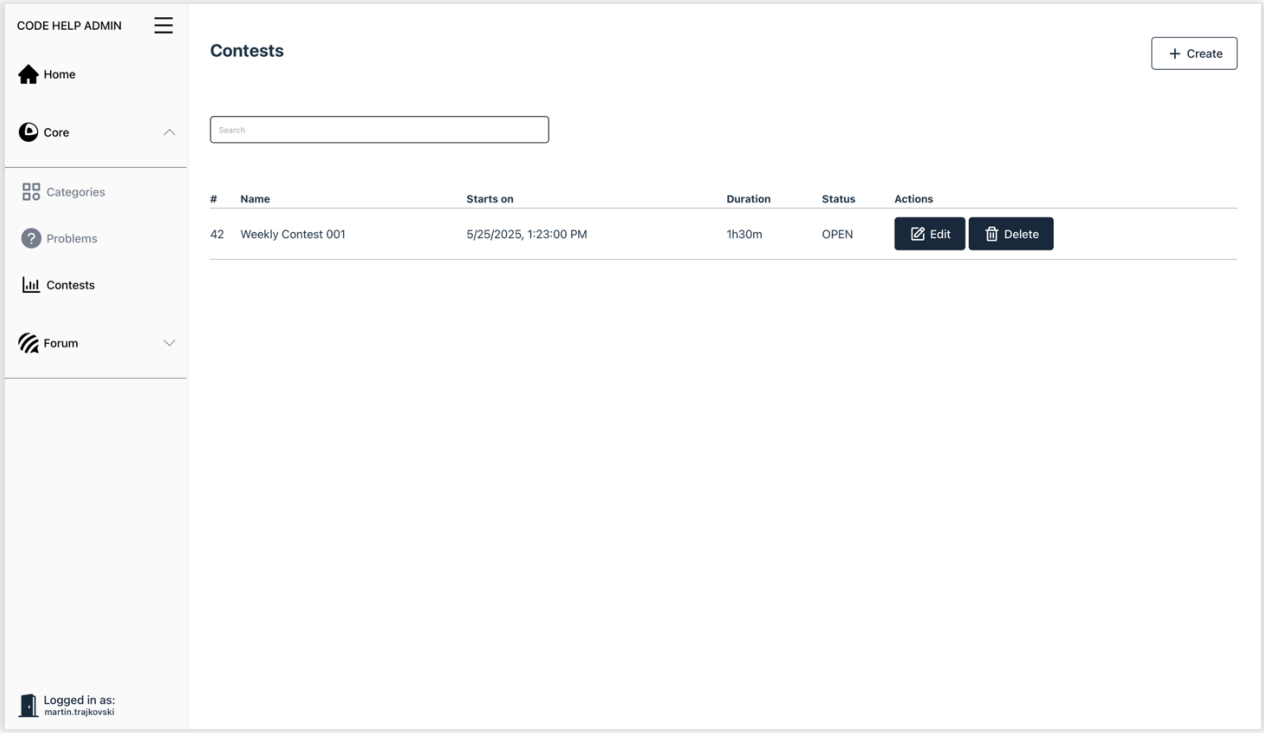


Слика 22 Најава во админ апликација

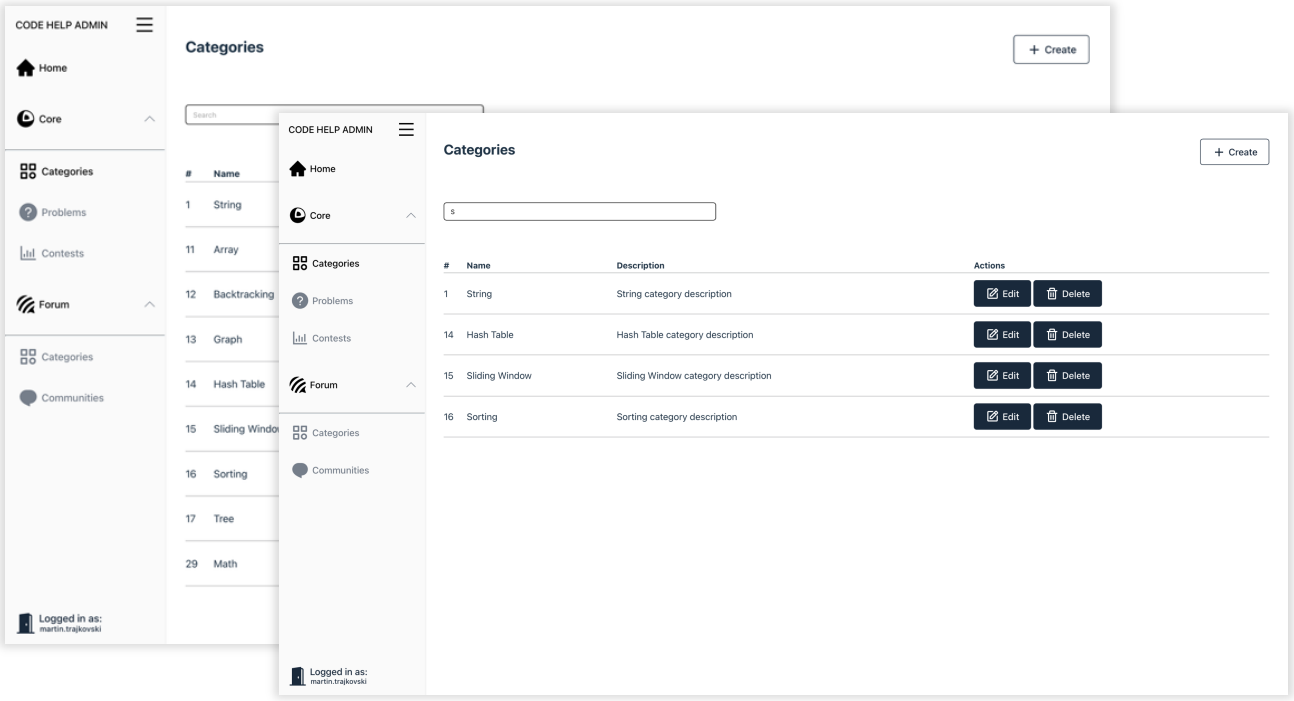
Дизајнот на корисничкиот интерфејс е насочен кон јасност и лесна употреба, со што се намалува можноста за грешки при администрирање и се подобрува целокупното корисничко искуство.

Филтрирање и преглед на податоци

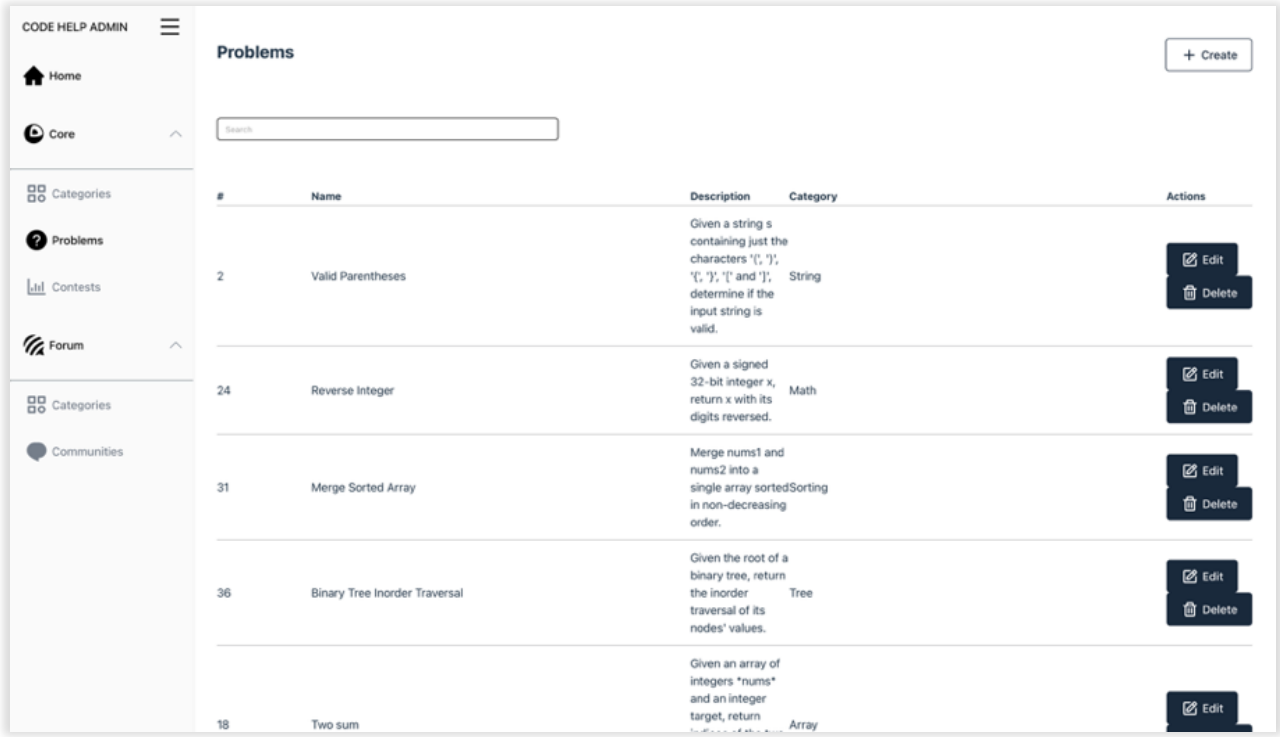
Филтрирањето и прегледот на податоците има сличен изглед и функционалност за сите ресурси во апликацијата со цел да се обезбеди конзистентност во корисничкото искуство. Се состои од една табела во која се листаат податоците кои се земаат од сервис поврзан со базата на податоци и се овозможува пагинација. За прегледот на податоците и нивно прикажување се земени минимални податоци со цел да не се оптоварува мрежата, клиентот и сервисот со враќање на големи податоци и негово преоптоварување. Во последната колона од табелата има акции за ажурирање или бришење на податокот.



Слика 23 Преглед на натпревари



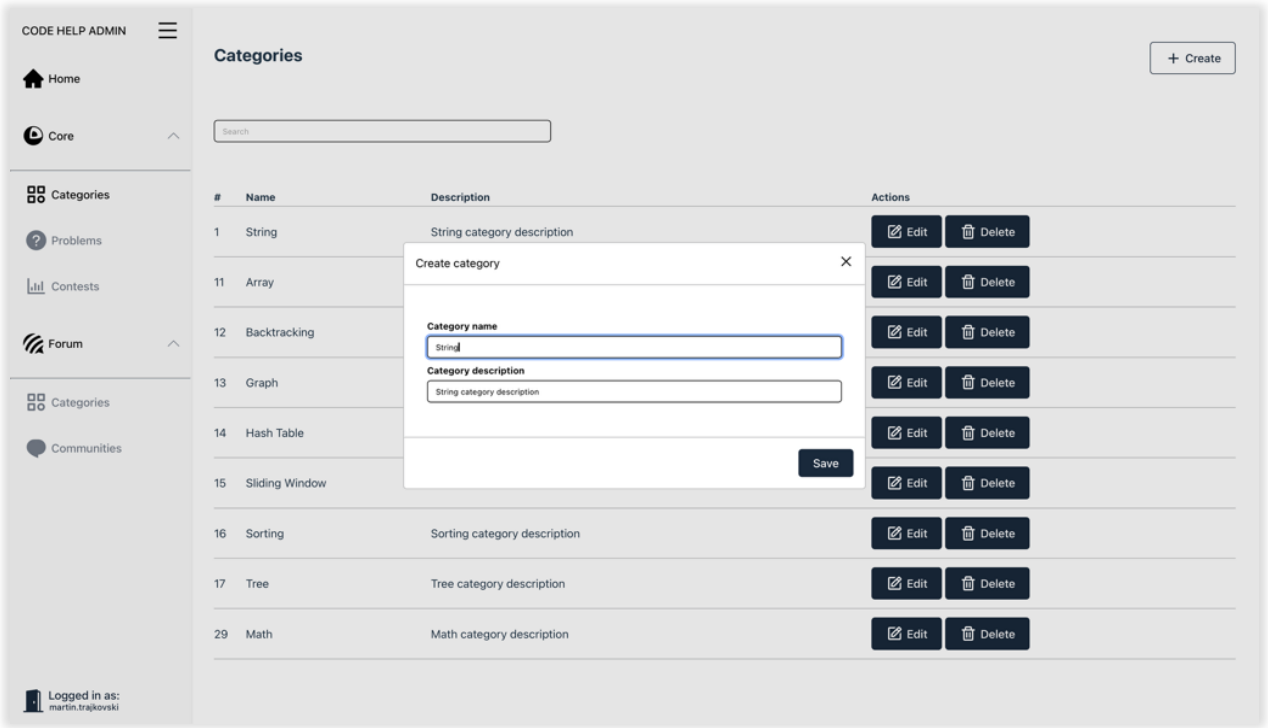
Слика 24 Преглед на категории и филтрирање



Слика 25 Преглед на задачи

Креирање и ажурирање на категории

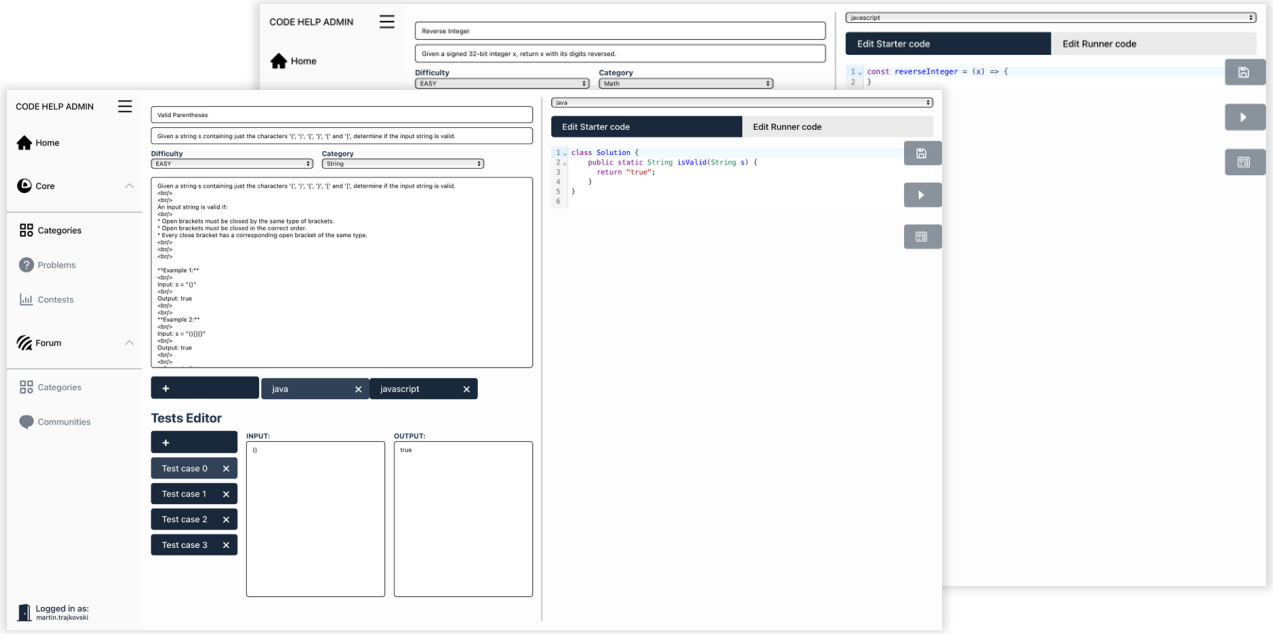
Како што може да се види на наредната слика, категорија може да се креира со клик на копчето „Create”. Се појавува модал во кој треба да се внесат податоците потребни за креирање на категорија, а потоа се зачувува со клик на копчето „Save”, или со притискање на Enter. При негово зачувување прво се праќа барање кон соодветниот сервис со што ентитетот се валидира и се зачувува во соодветната база на податоци. Кога успешно ќе се зачува категоријата, податоците се ажурираат со креирање на ново барање за земање на податоците.



Слика 26 Креирање на категории на задачи

Креирање на задачи

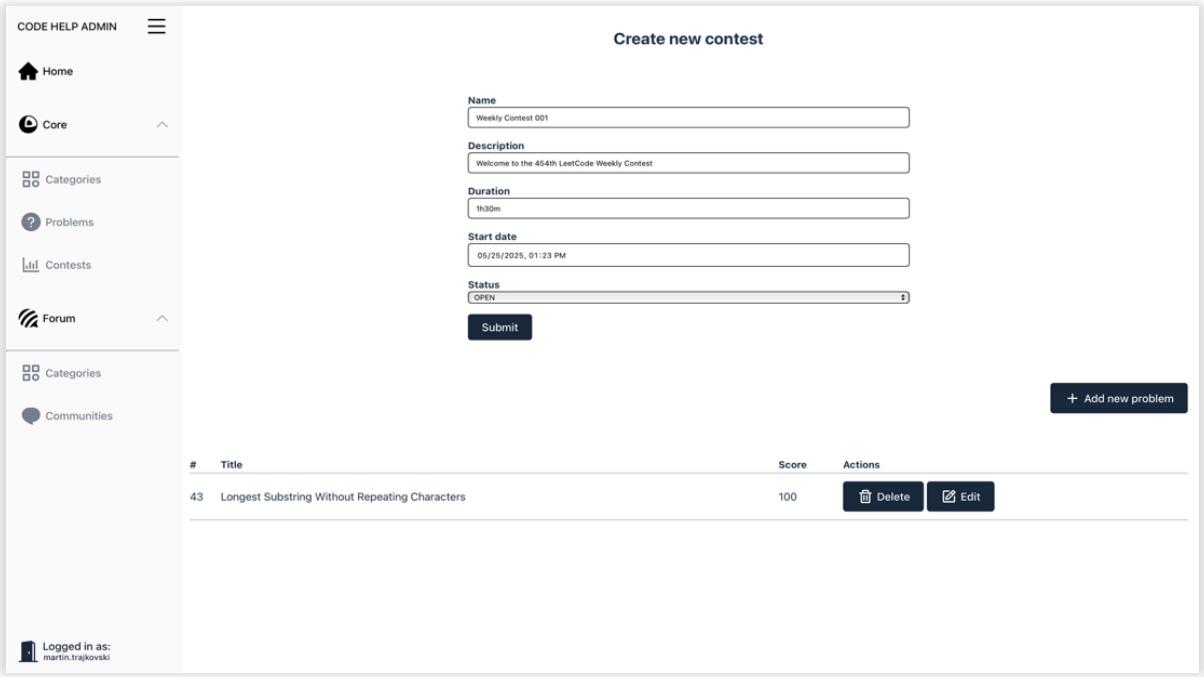
За да се креира задача прво треба да се внесат неколку податоци. Насловот на задачата, краток опис при преглед, тежина, категорија на задачата, тест примери и кодови на различни програмски јазици. Пред да се зачува задачата, може да се направи проверка со решение дали сите тест примери се во ред. Креираната задача може да се зачува со притискање на копчето за зачувување кое се наоѓа на десната страна на прозорецот.



Слика 27 Креирање на задача

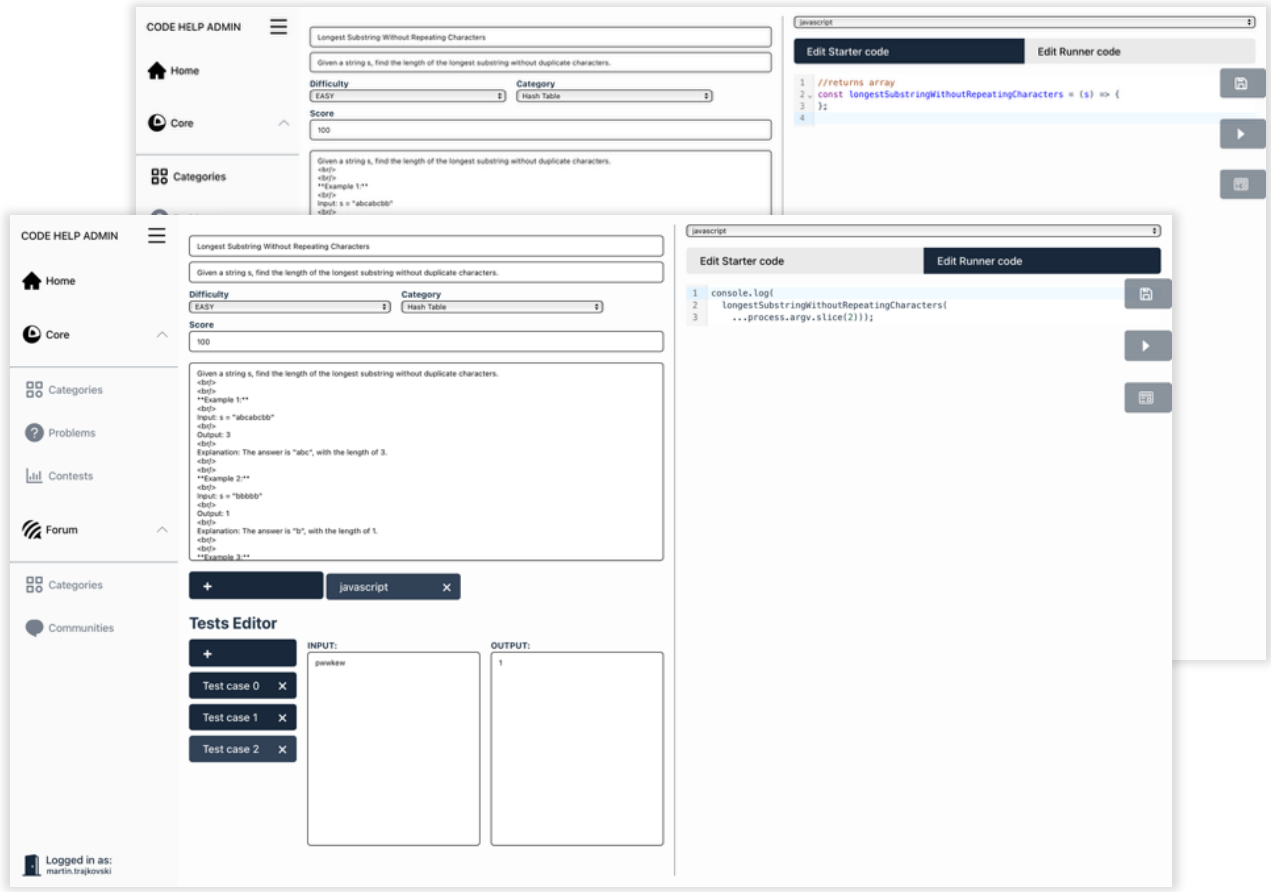
Креирање на натпревари

За да се креира еден натпревар треба да се внеси име на натпреварот, опис, трајност (колку време трае натпреварот откако ќе започне), момент на започнување на натпреварот и почетен статус на натпреварот кој се креира.

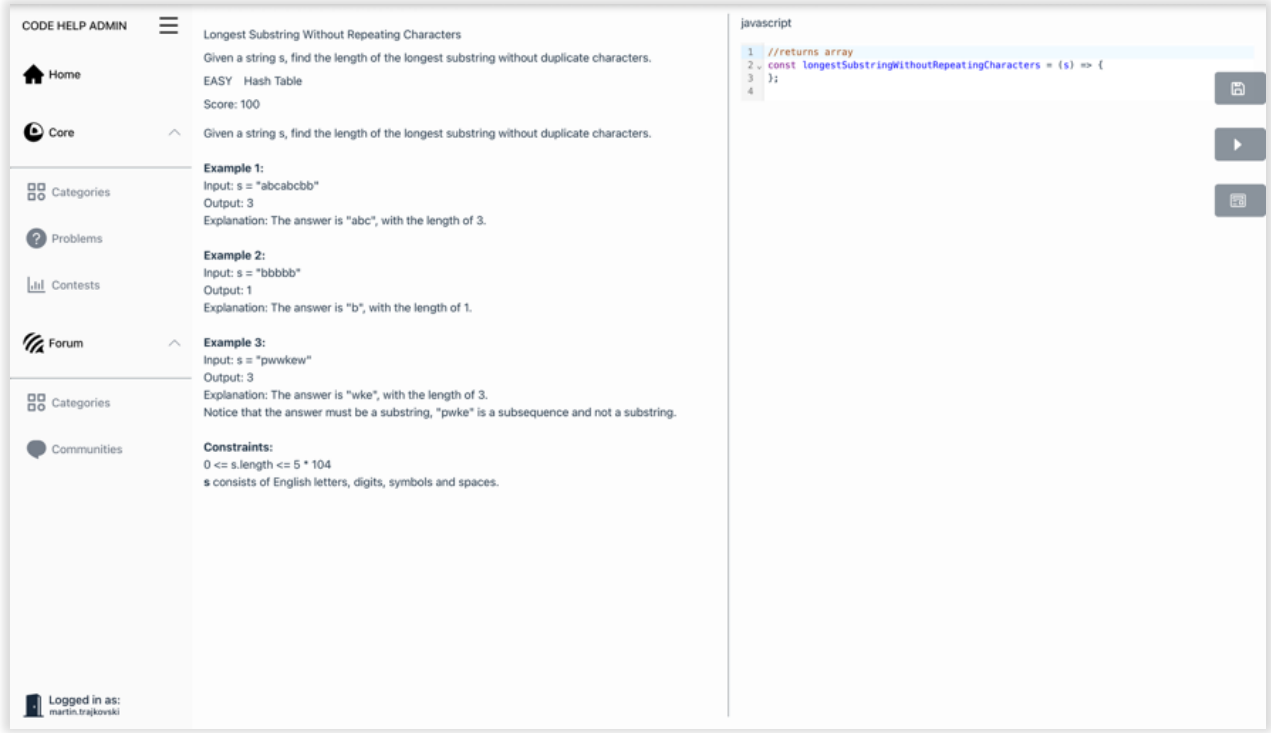


Слика 28 Креирање и ажурирање на наптревар

За креирање на задачи за натпреварот, тоа може да се направи со клик на копчето „Add New Problem”. При креирање на задача се внесуваат истите податоци кои една задача треба да ги има заедно со поените кои оваа задача може да ги донесе при нејзино решавање.



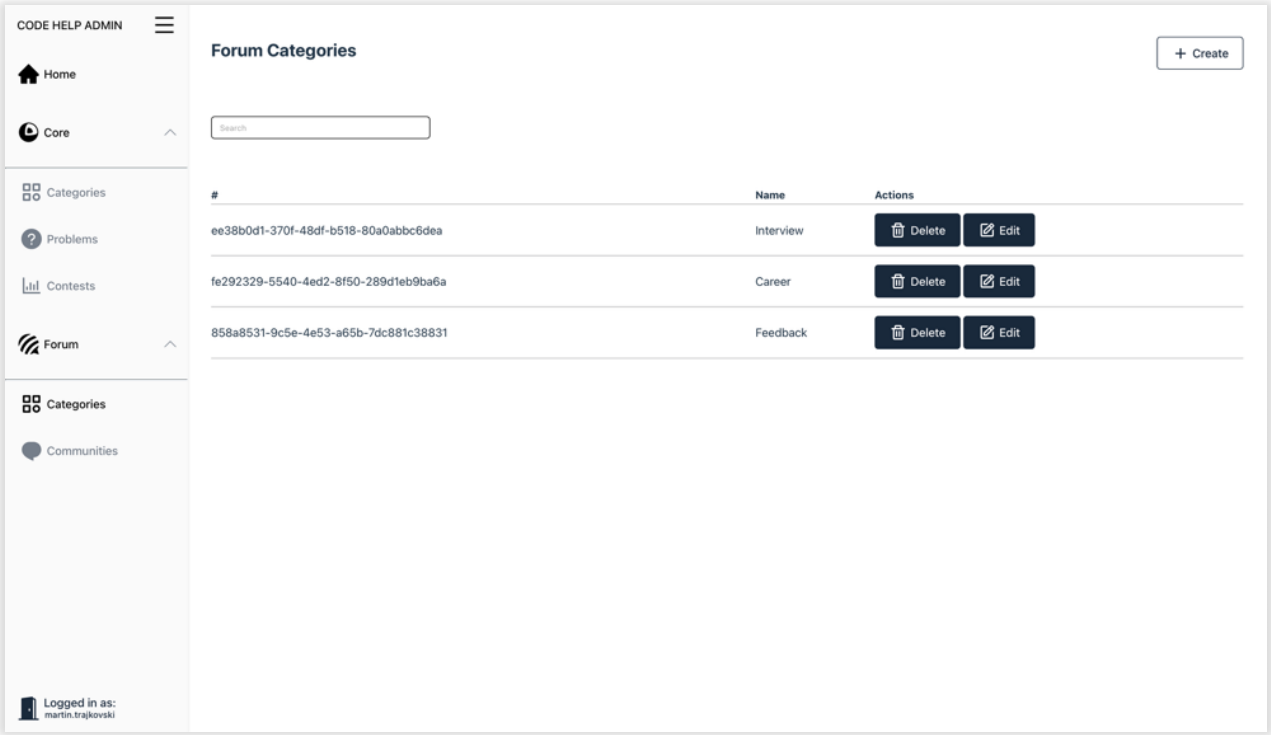
Слика 29 Креирање на задача за натпревар со преглед при креирање



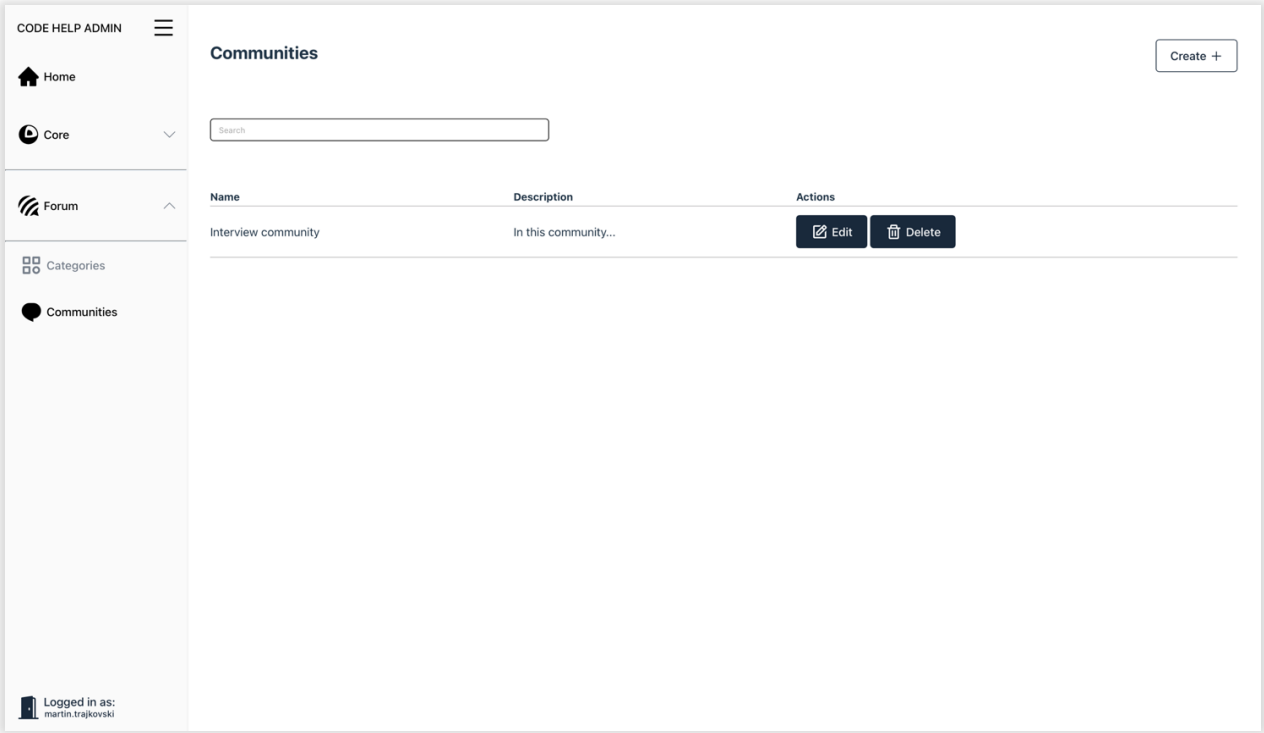
Слика 30 Преглед при креирање на задачи

Преглед на форум податоци и нивно креирање

Прегледот на податоците за форумот е сличен како и на другите слики за модулот за решавање на задачи и натпревари и се состои од слични функционалности за филтрирање, додавање на ресурс, ажурирање и бришење со цел да се задржи конзистентноста на изгледот во апликацијата.



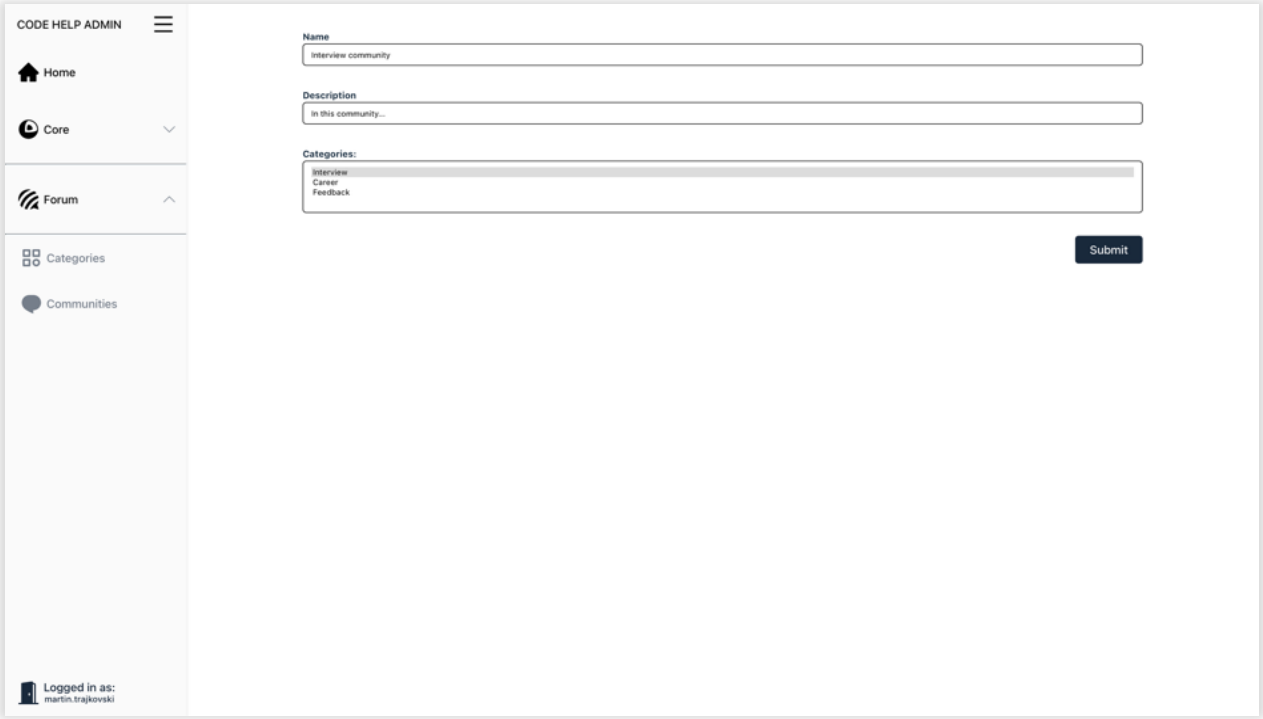
Слика 31 Прегчед на форум категории



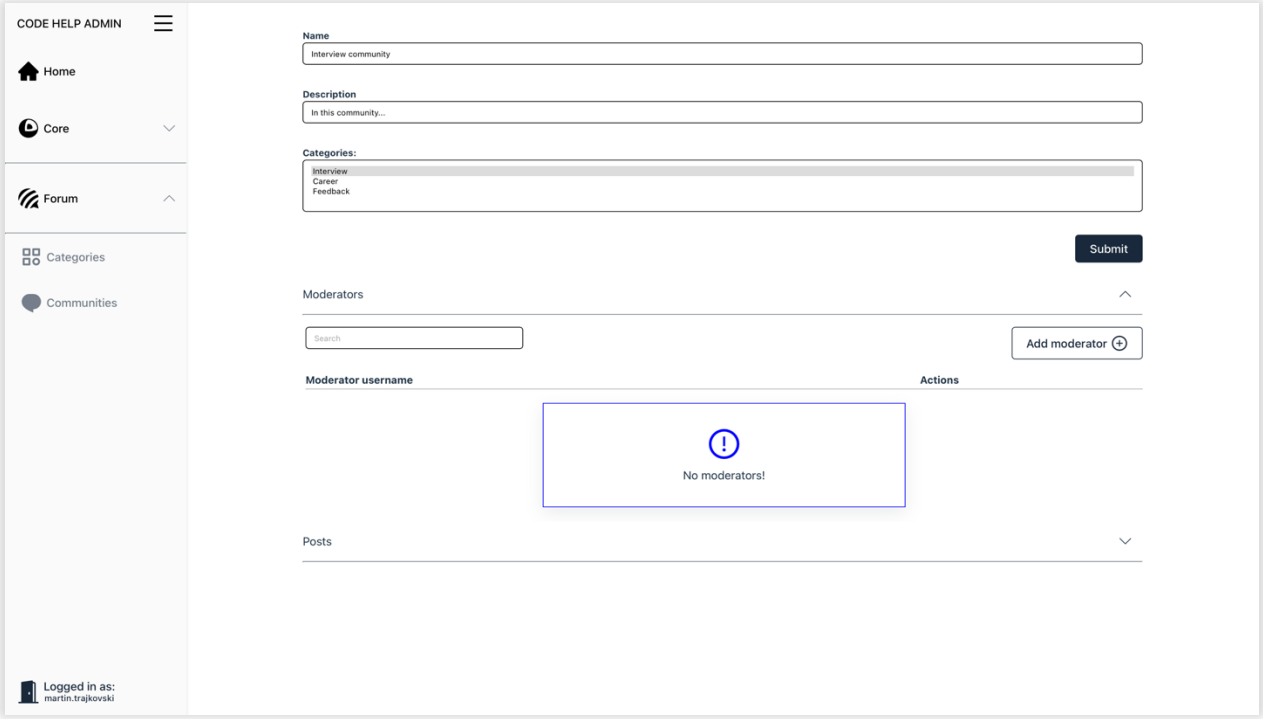
Слика 32 Преглед на заедници

Креирање на заедница

За креирање на една заедница преку админ апликацијата, најпрво треба да се креира форум категорија ако веќе не била претходно креирана. Потоа креираната категорија се избира при креирање на заедницата, се избира име на заедницата и краток опис за што служи оваа заедница кој ќе може да го видат крајните корисници за на крај да одлучат дали би сакале да се дел од оваа заедница.

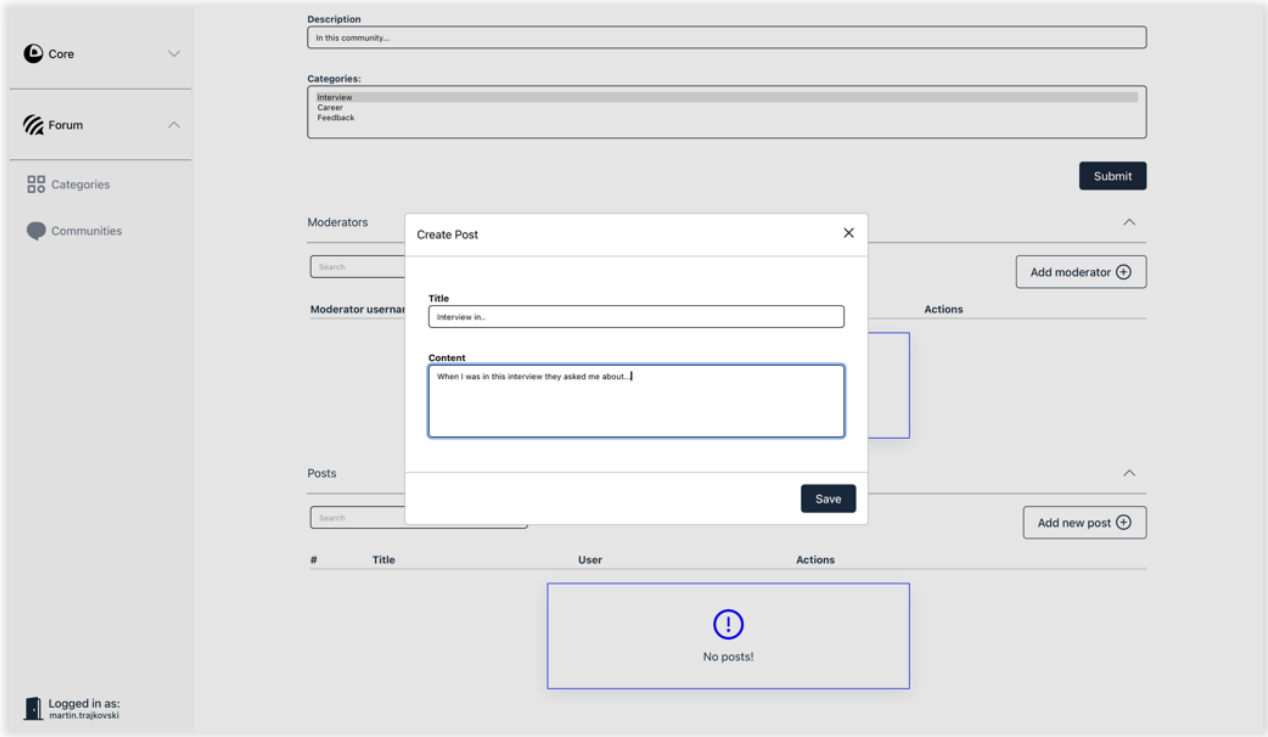


Слика 33 Креирање на заедница

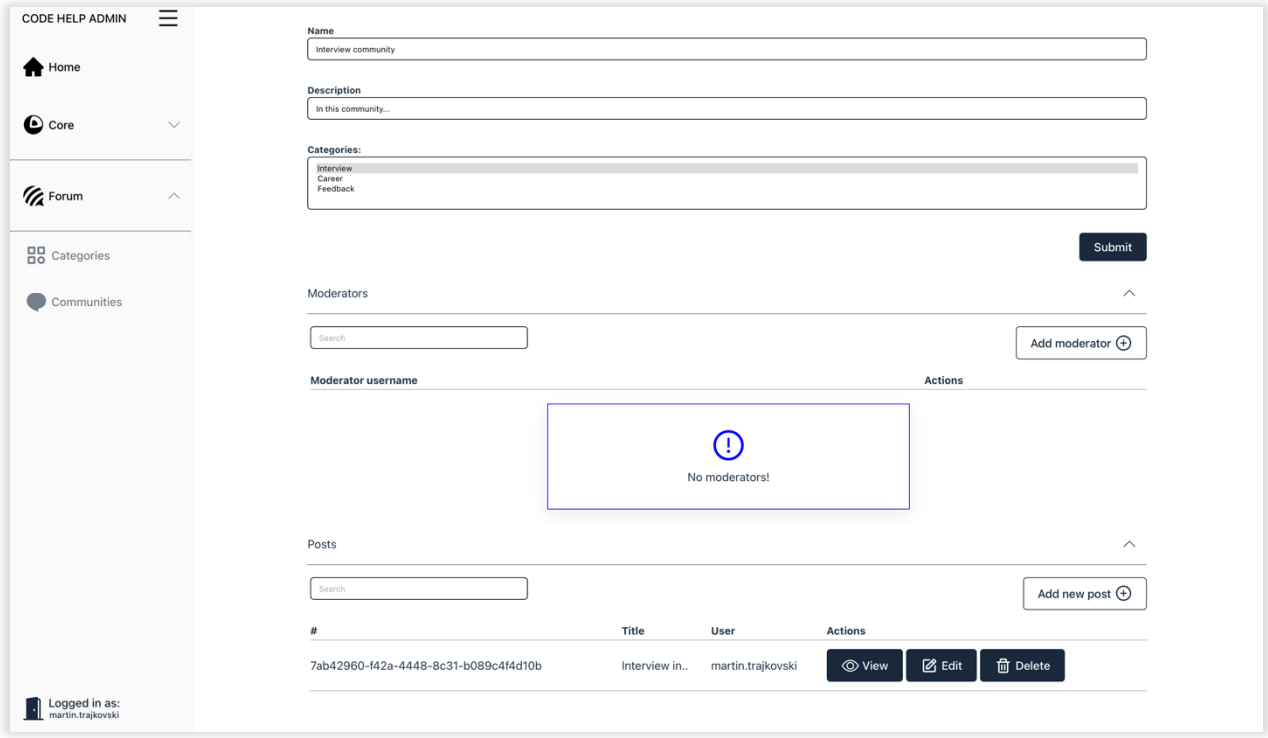


Слика 34 Креирање на заедница

Откако заедницата ќе биде успешно креирана таа се зачува во базата на податоци така што се праќа барање за зачувување до сервисот поврзан со апликацијата, а потоа за истата таа заедница администраторот може да избери корисници кои би биле модератори на оваа заедница и исто така да креира постови кои ќе може да ги видат крајните корисници и да коментираат на истите.



Слика 35 Креирање на постови од админ апликација



Слика 36 Преглед на заедница со креираниот пост

Референци

[1] VMware, “Spring Boot,” VMware, April 2014. [Мрежен]. Available: https://spring.io/projects/spring-boot. [Пристапено на 2026].

[2] Google, “Flutter,” Google, May 2017. [Мрежен]. Available: https://flutter.dev/. [Пристапено на 2026].

[3] Google, “Dart,” Google, 10 October 2011. [Мрежен]. Available: https://dart.dev. [Пристапено на 2026].

[4] VMware, “Spring,” VMware, June 2003. [Мрежен]. Available: https://spring.io. [Пристапено на 2026].

[5] J. Gosling, “Java Documentation,” Sun Microsystems, Oracle, May 1995. [Мрежен]. Available: https://docs.oracle.com/en/java/. [Пристапено на 2026].

[6] G. King, “Hibernate,” May 2001. [Мрежен]. Available: https://hibernate.org. [Пристапено на 2026].

[7] R. T. Fielding, M. Nottingham и J. Reschke, “RFC 9110 - HTTP Semantics,” June 2022. [Мрежен]. Available: https://datatracker.ietf.org/doc/html/rfc9110. [Пристапено на 2026].

[8] Red Hat, “Keycloak,” Red Hat, September 2014. [Мрежен]. Available: https://www.keycloak.org. [Пристапено на 2026].

[9] D. Hardt, “RFC 6749 - OAuth2.0,” rfceditor, October 2012. [Мрежен]. Available: https://datatracket.ietf.org/doc/html/rfc6749. [Пристапено на 2026].

[10] OpenID Foundation, “How OpenID connect works,” OpenID Foundation, 26 February 2014. [Мрежен]. Available: https://openid.net/developers/how-connect-works/. [Пристапено на 2026].

[11] B. Campbell, C. Mortimore и M. B. Jones, “RFC 7522 - SAML 2.0,” rfceditor, May 2015. [Мрежен]. Available: https://datatracker.ietf.org/doc/html/rfc7522. [Пристапено на 2026].

[12] M. B. Jones, J. Bradley и N. Sakimura, “RFC 7519 - JSON Web Token,” rfceditor, May 2015. [Мрежен]. Available: https://datatracker.ietf.org/doc/html/rfc7519. [Пристапено на 2026].

[13] I. S. Vladimirovich, “nginx,” October 2004. [Мрежен]. Available: https://nginx.org. [Пристапено на 2026].

[14] E. Rescorla, “RFC 2818 - HTTP Over TLS,” May 2000. [Мрежен]. Available: https://datatracker.ietf.org/doc/html/rfc2818. [Пристапено на 2026].

[15] R. Griesemer, R. Pike и K. Thompson, “The Go Programming Language,” Google, November 2009. [Мрежен]. Available: https://go.dev. [Пристапено на 2026].

[16] S. Hykes, “Docker,” Docker Inc. (dotCloud Inc.), 20 March 2013. [Мрежен]. Available: https://www.docker.com. [Пристапено на 2026].

[17] R. Harris, “Svelte,” 26 November 2016. [Мрежен]. Available: https://svelte.dev. [Пристапено на 2026].

[18] B. Eich, “JavaScript MDN,” May 1995. [Мрежен]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript. [Пристапено на 2026].

[19] Neo4j Inc., “Neo4j,” Neo4j Inc., February 2010. [Мрежен]. Available: https://neo4j.com. [Пристапено на 2026].

[20] P. M. Stonebreaker, “PostgreSQL,” 29 January 1997. [Мрежен]. Available: https://www.postgresql.org. [Пристапено на 2026].

[21] M. Belshe, R. Peon и M. Thomson, “RFC 7540 - HTTP/2,” rfceditor, May 2015. [Мрежен]. Available: https://datatracker.ietf.org/doc/html/rfc7540. [Пристапено на 2026].

[22] T. Bray, “RFC 8259 - JSON Data Interchange Format,” rfceditor, December 2017. [Мрежен]. Available: https://datatracker.ietf.org/doc/html/rfc8259. [Пристапено на 2026].

[23] M. Kelly, “JSON Hypertext Application Language,” 19 October 2023. [Мрежен]. Available: https://www.ietf.org/archive/id/draft-kelly-json-hal-11.html. [Пристапено на 2026].

[24] J. c. adjanohoun, “Data Structures and Algorithms Overview: The Core Pillars of Problem Solving,” Medium, 3 June 2025. [Мрежен]. Available: https://medium.com/@johnadjanohoun/data-structures-and-algorithms-overview-the-core-pillars-of-problem-solving-c44def4d15c7. [Пристапено на 2026].